

Hair as Memory: Retrieval Bottlenecks, Structural Gains, and Experimental Appendix

Sohan Poudel
forreserachonlyso@gmail.com
ORCID: 0009-0004-5762-3898

May 26, 2026

Transparency and Provenance Note

This manuscript is an exploratory research draft and should not be interpreted as a peer-reviewed claim of novelty, priority, or superior performance over alternative approaches. Its purpose is to document a diagnostic experimental program on latent reasoning control.

Large language models, including ChatGPT, Gemini, and Grok, assisted with brainstorming, drafting, editing, code scaffolding, and result interpretation. The experimental direction, code execution, result selection, and final framing were determined and verified by the author.

Readers should therefore treat this paper as an early proof-of-concept report whose empirical claims depend on the included synthetic experiments and should be independently verified before they are treated as established scientific conclusions.

For additional transparency, the author also provides supporting materials, including chat conversations and code used to run the experiments.

- Original ChatGPT conversation: <https://chatgpt.com/share/69fb9c59-6a90-8324-a7b5-e8877069364e>
- Original Gemini conversation: <https://gemini.google.com/share/6ea5706b5f64>
- Original Grok conversation: https://grok.com/share/bGVnYWN5LWNvcHk_3bccc0c1-8faa-4262-9e8b-d9a06d7c625a
- Colab notebook: <https://colab.research.google.com/drive/1XW4GvLdr6K2mFZiZyYxGDicYf2kvh6IS?usp=sharing>

Abstract

This paper studies whether a hair-style memory architecture can use structured placement to retrieve the right item more effectively than a strong flat content-addressed baseline. Across a long sequence of synthetic experiments, the main positive result is that multi-root assignment and larger root shortlists substantially improve candidate coverage. The main negative result is that this stronger candidate exposure still does not reliably translate into better end-to-end retrieval than flat key matching, because the final within-pool selector remains too weak and current spiral root maps are often too concentrated. The later experiments sharpen both points. A PCA-aligned spiral map is the first spiral variant in these experiments that becomes reasonably competitive in a favorable latent-manifold regime, and QDMN-style refinement helps mainly in low-margin ambiguous cases, but neither effect overturns the broader ranking against

random placement and strong flat baselines. A final matched-budget scale sweep makes the caution sharper: the current implementation does not yet validate the architecture at scale. The paper therefore argues for a narrower but more defensible conclusion: retrieval geometry matters because it shapes candidate access, but practical success still depends on better root lifting and a stronger ambiguity-aware within-pool scorer.

1 Problem and Claim

This paper studies a simple question: can a hair-like memory system use structured placement to retrieve the right item more effectively than a flat content-addressed baseline? In the model, each follicle stores a *root summary*, which acts like a compressed retrieval key, and a *shaft content*, which stores the detailed memory. A global controller—the “scalp”—first searches the root summaries and then reads out the selected shaft.

The central claim is now narrower and clearer than in earlier drafts. Structured placement does sometimes help, but only once retrieval explicitly exposes local or multi-root structure. The larger lesson is that candidate discovery has improved substantially, whereas the remaining end-to-end failure is concentrated in the final judgment stage: the system often surfaces a good pool and then fails to choose correctly inside it. The main text therefore emphasizes the current mechanism, the clearest empirical takeaways, and the updated bottleneck diagnosis. Detailed run-by-run history is retained in the appendix.

2 Method

We model memory item i with a key k_i and a value v_i . The root summary is a compressed projection

$$r_i = \text{normalize}(Pk_i),$$

where P is a shared random projection matrix. At query time, the controller projects the query into root space, scores every follicle with cosine similarity, and retrieves either the best follicle or the top- k follicles. The selected shaft, or weighted combination of shafts, is then decoded by nearest-neighbor matching in value space.

To test spatial robustness, each follicle is assigned a 2D position. When memory i is written, a leakage term spreads to nearby follicles:

$$x_j \leftarrow \text{normalize}\left(x_j + \lambda \exp(-\|p_i - p_j\|^2/(2\sigma^2))x_i\right),$$

where λ controls leak strength and σ controls locality. We compare two placement rules: a deterministic Fermat spiral and a random placement in the same disk.

Across the current series, the experiments use 256 synthetic memories, key and value dimension 64, noisy query trials, and multiple seeds. The main variants now fall into four groups: root-capacity sweeps that change the compressed key dimension, locality-aware retrieval rules that rerank candidates using spatial neighborhoods, multi-root or overlap mechanisms that enlarge the accessible candidate pool, and judge-stage variants that try to score candidates inside that pool more effectively. The key comparison throughout is not only spiral versus random placement, but also hair-style retrieval versus strong flat key nearest-neighbor baselines and oracle controls.

3 Results and Interpretation

The current results support five main takeaways.

Table 1 gives the full run-by-run map near the beginning of the paper, so a reviewer can see the whole project arc before reaching the appendix details. The main story is still the same: retrieval coverage is the strongest lever, geometry matters because it shapes coverage, spiral placement becomes competitive only when the projection is good, and QDMN helps mainly as a last-mile refinement in ambiguous cases.

#	Experiment name	What we did	What we got / what it means
1	M-CLTR controller baseline	Added continue, overwrite, and restart on a small synthetic memory task.	Accuracy reached about 0.89. The controller helped, but the main win was still retrieval quality.
2	Overlap roots + tiny cache grid	Swept <code>overlap_k</code> , <code>topk_root</code> , <code>guard_margin</code> , <code>overwrite_alpha</code> , and cache size.	Overlap and larger root fan-out helped a lot. Cache barely changed anything.
3	Best low-noise setting check	Looked at the top rows from the grid.	Some settings reached nearly perfect accuracy at low noise. That showed the pipeline can work, but the task was too easy there.
4	Paired best-recipe benchmark	Fixed the best retrieval recipe and used the exact same query bank for every method.	Many methods collapsed to similar behavior. This showed the judge was not yet the main bottleneck in easy settings.
5	Hard-noise fixed-recipe test	Kept retrieval fixed and increased noise.	Accuracy dropped quickly as noise rose. Robustness to noise became the main stress point.
6	Controller / guard sweep	Swept guard margins and overwrite strengths on the hard setting.	Gains were small. Adaptive control helped a bit, but not enough to be the main story.
7	Confusion benchmark	Increased group and cluster overlap to force ambiguity.	This created the first setting where judge quality started to matter more.
8	Hard-negative training	Trained judges using hard negatives drawn from the candidate pool.	Learned judges improved somewhat, but not enough to close the oracle gap.
9	Overlap / top-<i>k</i> sweep under hard noise	Tested overlap and root fan-out while keeping the hard benchmark fixed.	More overlap and larger <code>topk_root</code> improved coverage strongly. Retrieval coverage was confirmed as a major lever.
10	Controller sweep under hard noise	Compared guarded, adaptive, and QDMN-style controllers on the same hard setting.	Adaptive control sometimes helped, but the gains stayed small and unstable.
11	Ambiguity / margin-binned test	Split queries by judge margin into low-, mid-, and high-ambiguity bins.	QDMN helped most in low-margin cases, which makes it a last-mile refinement tool rather than a global fix.
12	Best method per ambiguity bin	Checked which method wins in each margin bin.	No method dominated everywhere. The oracle remained above everything, and practical methods became close only when the query was easy.
13	Placement debug: random vs. spiral_{ortho}	Measured root spread, nearest-neighbor cosine, and root assignment entropy.	Random covered space better than the bad spiral variants. The projection was the main problem, not the idea of structure itself.
14	Case sweep across data regimes	Tested separated, clustered, shared-core, and latent-manifold data.	Random usually won overall. Spiral was sometimes competitive, but not consistently better.
15	Spiral geometry refinement	Added <code>spiral_pca</code> and compared it against <code>spiral_bad</code> and <code>spiral_ortho</code> .	<code>spiral_pca</code> became the best spiral version and the first genuinely competitive one.
16	Full case + placement + noise sweep	Ran all cases and placements across the full noise range.	The cleanest overall story emerged: random stayed best on average, <code>spiral_pca</code> was the strongest spiral, and QDMN helped mainly in the ambiguous tail.
17	Final QDMN gain check	Compared QDMN against flat and guarded methods only in the hardest ambiguous queries.	QDMN gave small but real gains in low-margin bins. That supports a last-mile refinement claim, not a replacement-for-retrieval claim.
18	Final matched-budget scale sweep	Scaled the strongest current recipe from 256 to 2048 items under matched budget while comparing random and <code>spiral_pca</code> against flat, judge-based, adaptive, QDMN, and oracle methods.	The sharpest negative result arrived here: random stayed stronger than <code>spiral_pca</code> , practical judges remained tightly clustered, and runtime rose quickly. That weakens confidence in the current implementation more than it rules out the broader idea.

Table 1: Full project overview placed near the beginning of the paper. The table summarizes the eighteen major experiments behind the final claims before the reader reaches the appendix.

The table gives the full experimental map; the five takeaways below explain the highest-level lessons

that remain stable across that map.

1. Root quality is still the first-order constraint. Across both the early sweeps and the later diagnostics, performance rises strongly with root dimension and falls sharply with query noise. When leak is removed, spiral and random placement become essentially identical, which shows that the dominant early failure is not geometry by itself but information loss in the compressed root. The later oracle-control split makes this even clearer: passive top-1 remains near 0.25–0.26 item accuracy in the main setting, the candidate oracle jumps to about 0.52, and the local oracle rises further to about 0.55. Root compression and root ranking therefore account for the largest share of lost accuracy, while neighborhood selection contains a smaller but still recoverable contribution.

2. Structured placement changes geometry reliably, but only weakly improves retrieval unless the search rule uses that structure directly. The spiral layout consistently produces cleaner spacing and lower interference than random placement. That structural advantage is real, but most passive retrieval rules do not convert it into much end-to-end gain. The first clear signal appears only once retrieval becomes locality-aware: the v3 two-stage reranker gives spiral a small but repeatable edge over random, the v4 spatial-window variant removes that gap, and the v5 active-local follicle restores a modest spiral advantage under several settings. The right claim is therefore limited but meaningful: spatial regularity helps only when retrieval explicitly exposes local structure, and even then the effect remains modest.

3. Multi-root retrieval and overlap are the strongest practical advances so far. The biggest change in belief across the project is that candidate discovery is no longer the main weakness in the same way it was earlier. Query-based reranking already helps substantially, and the later multi-root union experiments are the largest practical jump: at the main operating point, retrieval rises to about 0.77 item accuracy, while the oracle union reaches about 0.90. Overlapping root assignment and larger root shortlists improve coverage further, with overlap $k = 3$ and top- $k = 10$ clearly outperforming narrower settings on average. These results make the architecture look more viable than the early single-root experiments suggested. The system can now expose a strong candidate set; it just does not yet score that set well enough.

4. The dominant unresolved problem has shifted to within-pool judgment. Once the candidate pool becomes strong, the practical system still usually trails a flat key nearest-neighbor baseline. In the main scale comparison, flat search reaches about 0.80 item accuracy while multi-root reranking remains near 0.77, even though the oracle union is far higher. Later judge-stage experiments all reinforce the same point. Soft local smoothing is at best marginally helpful, naive averaging is destructive, shallow learned judges do not close the gap, guarded variants improve stability but not enough strength, adaptive shortlist sizing helps only slightly, and controller-style overwrite or restart mechanisms do not materially change the ranking. The clearest updated conclusion of the paper is therefore that exposing better candidates is no longer enough: the system still lacks a stable, expressive scorer inside the discovered pool.

5. The remaining ambiguity is now about the selector, and the spiral story is still incomplete. The newer hard-confusion benchmark confirms that oracle-pool performance remains well above every practical judge, so the shortlist is often already good enough and the remaining failure is concentrated in selection under ambiguity. At the same time, the paired best-recipe evaluation surfaced an implementation warning: the current spiral root construction can perform dramatically worse than random in some settings, which makes any strong geometric claim premature. The follow-up Part 16B spiral-debug run strengthens that caution rather than removing it: an orthonormalized spiral construction recovers much of the earlier collapse relative to the old spiral map, but random placement still remains stronger on average across the ambiguity and controller

sweeps. Part 17 then sharpens the selector diagnosis one step further by holding retrieval fixed and binning cases by shortlist margin. The result is clarifying: when the pool already has a high margin, nearly every practical method behaves the same and retrieval is effectively solved; when the margin is low, the oracle gap remains large and adaptive or refinement-style methods help only modestly and inconsistently. Part 18 adds a broader placement sanity sweep across several synthetic regimes and makes the spiral result more concrete. Random placement is not merely slightly better on average; it also produces much less concentrated root geometry, more uniform root usage, and better coverage across every tested regime than either spiral construction, including the orthonormalized version. Part 19 refines that picture rather than overturning it. A PCA-aligned spiral map is the first spiral variant to become genuinely competitive, and in the latent-manifold regime it is actually the strongest placement overall. But the broader conclusion still holds: random remains the best average baseline across the separated, clustered, and shared-core regimes, while QDMN-style refinement delivers only small last-mile gains that appear mainly in the lowest-margin ambiguous slice rather than as a broad improvement. The paper can therefore end with a sharper but more cautious statement. Hair-style retrieval now has credible evidence for improved candidate access through multi-root and overlap mechanisms, but it still does not beat a strong flat baseline in practical end-to-end retrieval across the full sweep, and current spiral root maps remain architecture-sensitive rather than robustly superior. The most promising next step is not another minor heuristic tweak. It is a stronger ambiguity-aware within-pool judge, together with continued audit or redesign of the spiral construction.

A Experimental Appendix

This appendix preserves the detailed experimental record behind the main claims. The main text now foregrounds the stable conclusions: root compression remains the first bottleneck, multi-root and overlap mechanisms materially improve candidate access, and the remaining end-to-end gap is concentrated in within-pool judgment. The subsections below keep the narrower ablations, intermediate failures, and run-by-run implementation history that support those conclusions.

Each subsection still answers four questions for a run: *what* was tested, *how* it was implemented, *what* the result was, and *what we learned*. That format makes it possible to preserve the project’s full trajectory without forcing the main narrative to read like a lab notebook.

A.1 Run A: Leak Sweep

What was tested. This run tests whether a more regular placement geometry helps protect memory retrieval when writing one follicle leaks into nearby follicles.

How it was done. Root dimension was fixed at 16 and top- k retrieval was fixed at 1. Leak strength was swept from 0 to 0.35, and results were averaged over five seeds. Spiral and random placement were compared directly under the same leakage rule.

What happened. Figure 1 shows that both layouts degrade smoothly as leakage increases. At zero leak, spiral and random are identical, both reaching answer accuracy 0.138. At leak 0.35, random ends at 0.0945 while spiral ends at 0.0875.

What we learned. Spiral clearly improves geometry, but not yet final retrieval. The mean nearest-neighbor distance is 0.1052 for spiral versus 0.0590 for random, and the interference score is 0.0516 for spiral versus 0.0567 for random. So the geometry signal is real, but the current memory and decoding scheme is too weak to turn that structural advantage into better performance.

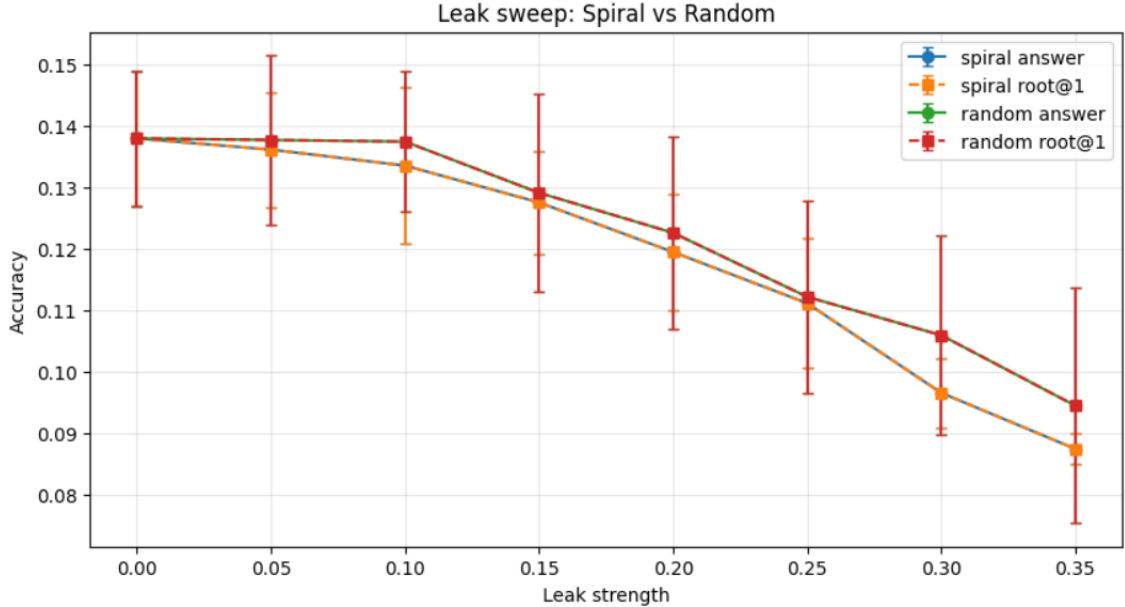


Figure 1: Leak sweep with fixed root dimension 16 and $\text{top-}k = 1$. Both layouts lose accuracy as leakage increases. Spiral has better geometry, but random remains slightly better in final retrieval accuracy under the current leakage-and-decoding rule.

A.2 Run B: Root-Dimension Sweep

What was tested. This run tests whether the main bottleneck is spatial placement or root compression.

How it was done. Leak was set to zero so that placement effects were removed. Root dimension was swept across 8, 16, 32, and 64, while all other settings were held fixed. Spiral and random layouts were both evaluated, again averaged across five seeds.

What happened. Figure 2 shows that spiral and random overlap exactly at every root dimension. Accuracy rises sharply with root size: 0.057 at 8, 0.138 at 16, 0.296 at 32, and 0.485 at 64.

What we learned. This is the cleanest result in the current study. Without interference, placement does not matter at all in the present code; only the root representation matters. That means the compressed root is the main failure point in the baseline system. A 16-dimensional root compressing a 64-dimensional key is simply too lossy for stable retrieval.

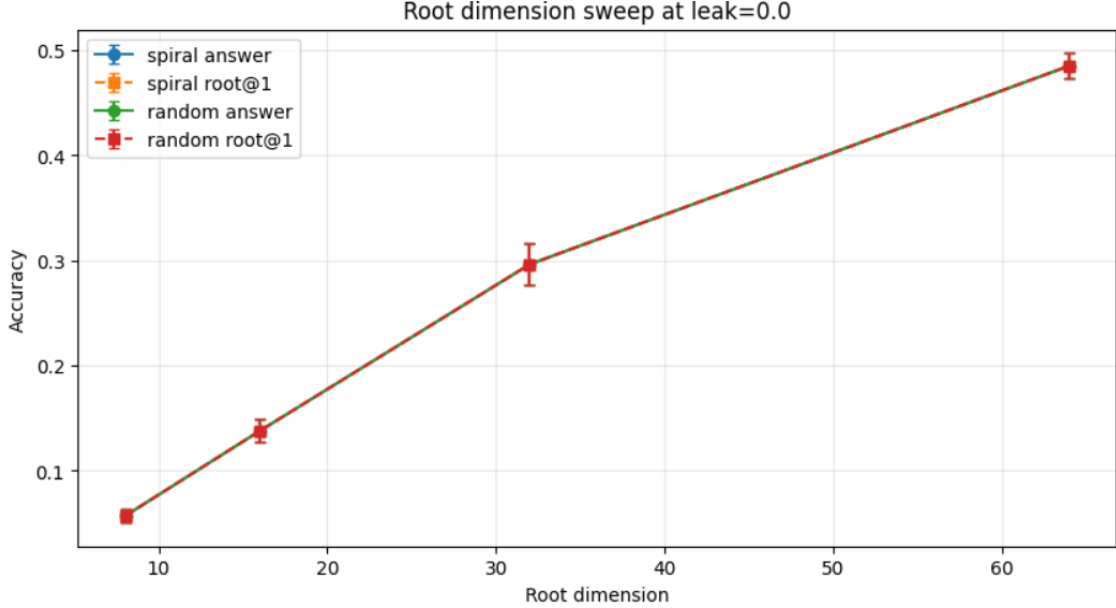


Figure 2: Root-dimension sweep at zero leak. Spiral and random overlap exactly, showing that placement has no effect when interference is removed. Accuracy is driven mainly by root compression quality.

A.3 Run C: Top- k Sweep

What was tested. This run tests whether retrieving several candidate follicles instead of only one makes the system less brittle.

How it was done. Leak was fixed at 0.10 and root dimension was fixed at 16. Retrieval was evaluated at $k = 1$, $k = 3$, and $k = 5$, using a soft combination of the retrieved shafts before decoding the final answer.

What happened. Figure 3 shows that root@ k rises strongly as k increases. For spiral, root@ k rises from 0.134 at $k = 1$ to 0.345 at $k = 5$; for random, it rises from 0.138 to 0.341. However, answer accuracy stays nearly flat or drops slightly: random falls from 0.1375 to 0.1297, while spiral moves from 0.1336 to 0.1313. Spiral does show a small answer-accuracy edge at $k = 3$, reaching 0.1354 versus 0.1302 for random.

What we learned. The system is often retrieving the correct neighborhood but decoding it badly. The likely reason is that the current soft voting rule averages multiple shafts into a mixed representation rather than selecting a clear winner. This run therefore points to decoding, not search, as the next bottleneck. It also hints that spiral may help once retrieval becomes less brittle.

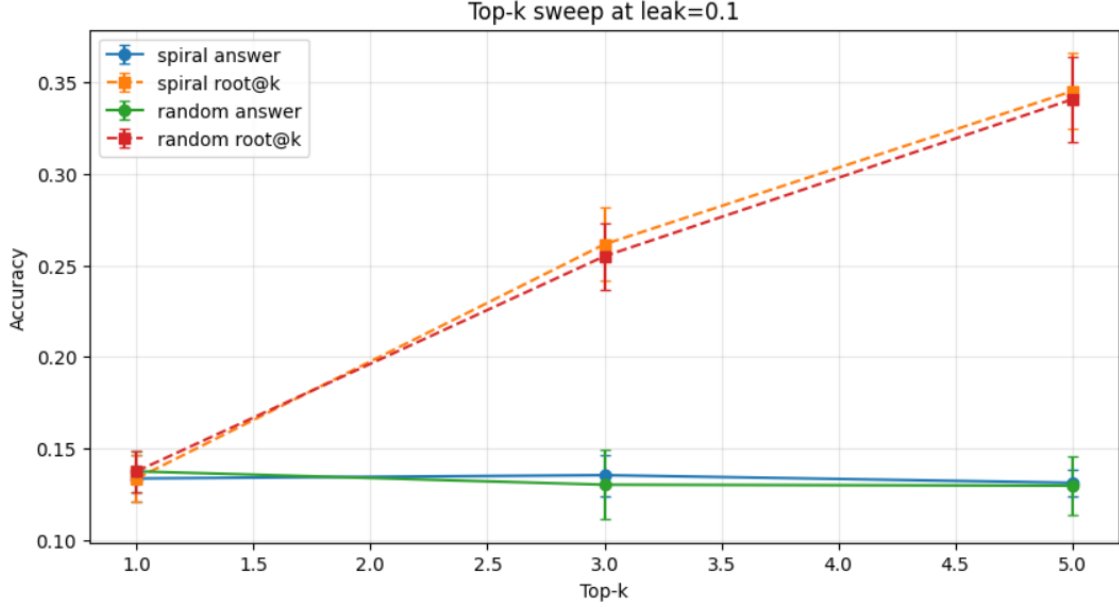


Figure 3: Top- k sweep at leak 0.10. Root@ k rises strongly with larger k , but final answer accuracy does not. This indicates that candidate retrieval improves while the current shaft-combination decoder remains a bottleneck.

A.4 Run D: Geometry Sanity Check

What was tested. This run checks whether the two spatial placement rules actually differ in the intended geometric way.

How it was done. We plotted the follicle coordinates for the deterministic Fermat spiral and for a random disk placement using the same number of items and radius.

What happened. The spiral layout shows a visibly more even packing, while the random layout shows local clumping and uneven spacing.

What we learned. The geometry intervention itself is working. Any failure of spiral to improve retrieval is therefore not because the placement rule failed to change the storage layout; it is because the downstream memory and decoding rules are not yet exploiting that structure strongly enough.

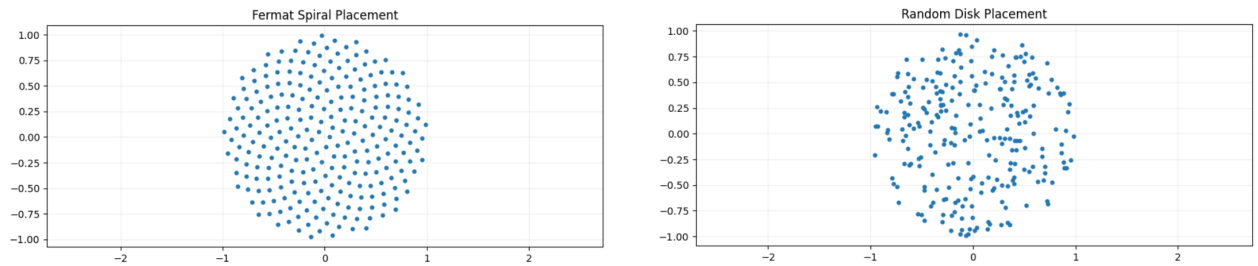


Figure 4: Follicle coordinates for the two placement rules. The Fermat spiral provides a more uniform spatial packing, which explains its larger nearest-neighbor distance and lower average interference.

A.5 Run E: Locality-Aware Two-Stage Retrieval (v3, merged rerun)

What was tested. This run asks whether spiral placement can matter *directly* in retrieval rather than only through leakage. The main idea is to create clustered memories, preserve their write-order locality, and then use a two-stage retrieval rule that explicitly looks at geometric neighbors. This subsection now combines the original v3 run with a later near-duplicate rerun whose plots are

cleaner but whose conclusions are essentially the same.

How it was done. Memories were organized into 64 semantic groups of size 4. Items in the same group were created by adding jitter around shared key and value centers, so that nearby memories in write order were semantically related. Retrieval used two stages: Stage A performed a global coarse search over root summaries, and Stage B reranked the top global candidates using support from their local geometric neighborhoods. Three sweeps were evaluated: local neighborhood size, root dimension, and query noise. In the refreshed rerun, the plotting was cleaned up and the baseline global top-1 curve was shown more explicitly, but the retrieval rule and qualitative outcome stayed almost unchanged.

What happened. The key pattern is that spiral finally shows a small but consistent retrieval advantage once locality is used explicitly. In the local-neighborhood sweep, spiral local rerank accuracy is higher than random at neighborhood sizes 3, 5, 9, and 15, and essentially tied at 25. At local neighborhood size 9, spiral reaches 0.2568 while random reaches 0.2471. The refreshed rerun makes the same point a little more clearly by showing the stage-1 global top-1 baseline directly: local reranking exposes a small spiral edge, but both reranked curves still sit below the stronger global baseline. In the root-dimension sweep, spiral again slightly exceeds random at root dimension 32 (0.2568 versus 0.2471) and 64 (0.4279 versus 0.4266). In the query-noise sweep, the two placements remain very close, with the rerun again showing that both layouts degrade similarly as noise rises.

What we learned. This is the first run where it is reasonable to say that structured placement helps retrieval a little once retrieval is made locality-aware. The rerun strengthens confidence in that claim because the same qualitative pattern survives minor implementation and plotting changes. However, the effect is still weak. The most important limitation is that the second-stage reranker often performs worse than the original stage-1 global top-1 result. For example, with local neighborhood size 9, global top-1 accuracy is 0.2747, while the reranked accuracy falls to 0.2568 for spiral and 0.2471 for random. That means the local neighborhood vote is exposing a real spiral signal, but it is also dragging performance down by averaging in unrelated neighbors. A likely reason is that the semantic group size is only 4, while the reranker often consults 9 or more geometric neighbors, so many irrelevant follicles are being allowed to influence the decision.

A careful summary of Run E is therefore: structured placement gives a small and consistent retrieval benefit once locality is used in the search rule, and a near-duplicate rerun confirms that this conclusion is not a plotting accident. At the same time, the current local jury is still too noisy. This points directly to the next experiment: tighten the spatial window or replace soft local voting with winner-takes-all routing.

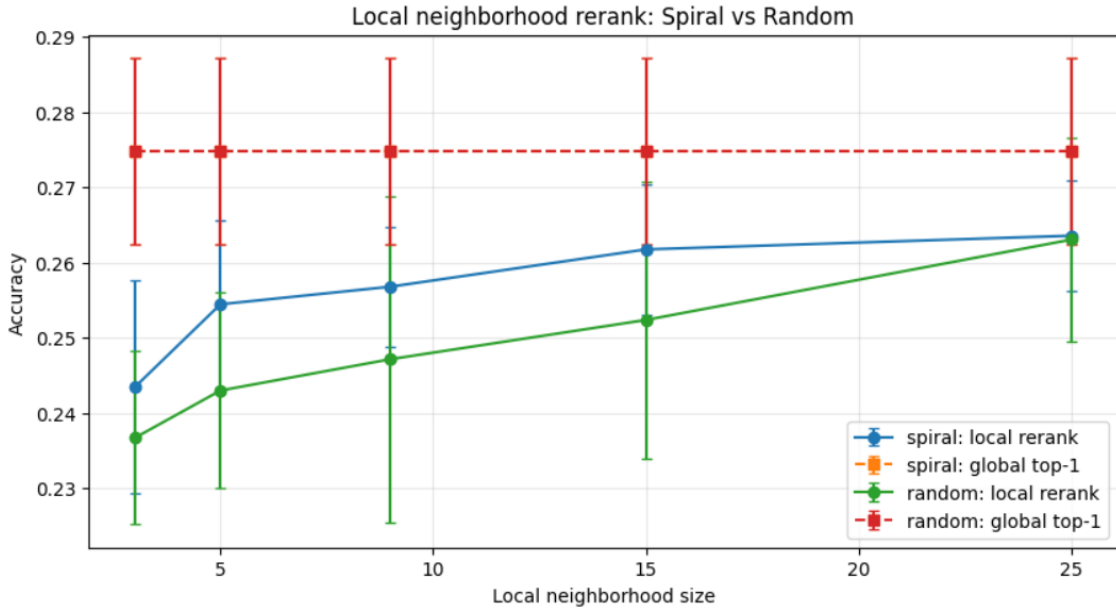


Figure 5: Run E merged local-neighborhood sweep. The refreshed rerun makes the comparison clearer by plotting both local rerank and stage-1 global top-1 baselines: spiral keeps a small rerank advantage over random across most neighborhood sizes, but the reranker still trails the stronger global baseline.

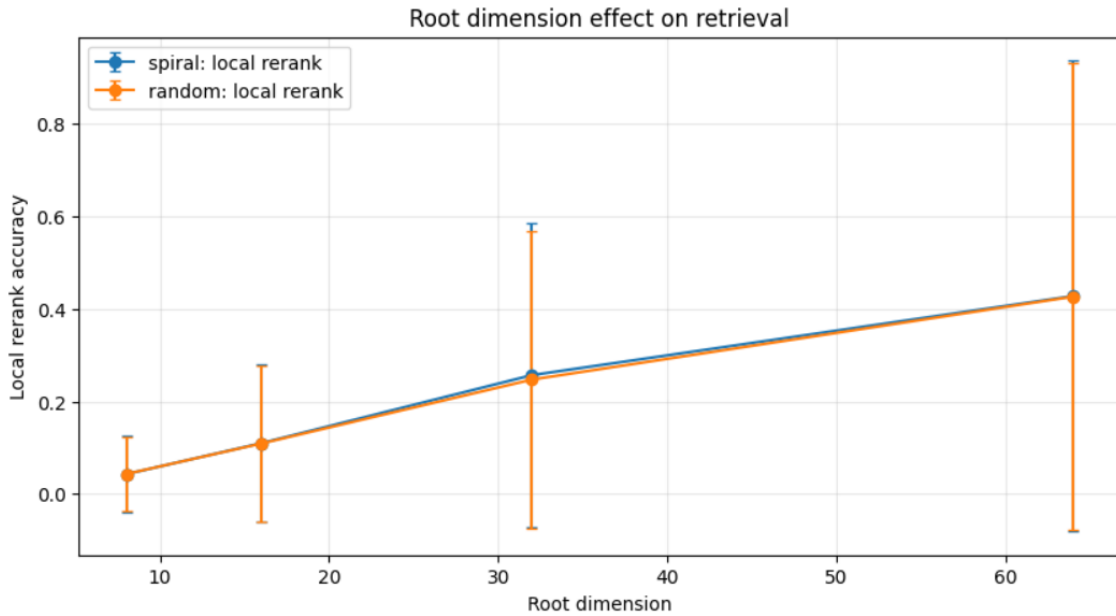


Figure 6: Run E merged root-dimension sweep. Root compression still strongly controls performance, while spiral remains only marginally above random under locality-aware reranking.

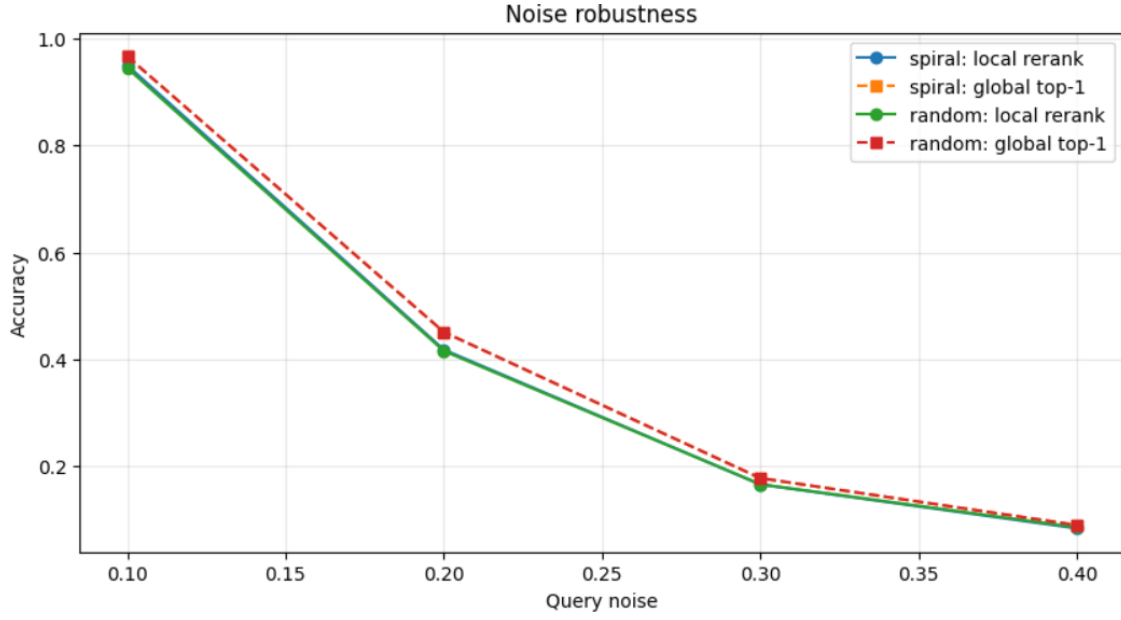


Figure 7: Run E merged query-noise sweep. Both layouts degrade in nearly the same way as noise increases, and the refreshed plot again shows that the locality-aware reranker does not surpass the underlying global top-1 baseline.

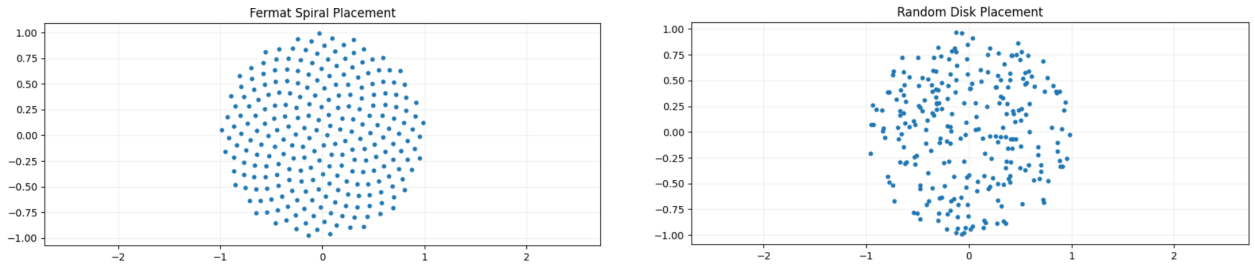


Figure 8: Run E placement geometry. The refreshed rerun uses the same two placement rules and again shows the underlying structural difference: spiral is more uniform, while random remains visibly clumpier.

A.6 Run F: Spatial-Window Retrieval (v4)

What was tested. This run asks whether a stricter spatial-window rule can make spiral placement matter more directly. Instead of using a soft local jury around several candidates, the system first selects a strong global candidate and then opens only a local spatial window around that candidate.

How it was done. Memories remained clustered as in Run E. Retrieval worked in two stages: Stage A performed global coarse search over root summaries, and Stage B restricted attention to a fixed spatial radius around the selected candidate. Three sweeps were evaluated: spatial window radius, root dimension, and query noise. The intention was that spiral placement would benefit from cleaner local structure, while random placement would suffer from noisier neighborhoods.

What happened. Figure 9 shows the main result immediately: spatial rerank accuracy is flat across all tested window radii, and the spiral and random curves overlap almost perfectly at about 0.2747. The same pattern holds in the root-dimension and query-noise sweeps. At root dimensions 8, 16, 32, and 64, spiral and random are numerically identical in spatial rerank accuracy. Likewise, across query noise values 0.1, 0.2, 0.3, and 0.4, the two placements again overlap exactly.

What we learned. This is an important negative result. The spatial-window version does not extract a useful spiral advantage; in practice it collapses to the stage-1 global top-1 behavior. The likely reason is that the windowed refinement is not adding real discriminative information beyond the best coarse candidate, so geometry is again present structurally but inactive functionally. In that sense, v4 strengthens the conclusion that passive geometric organization alone is not enough. If the project continues, the next step should probably be a more active follicle mechanism or a stronger learned routing rule rather than another passive locality heuristic.

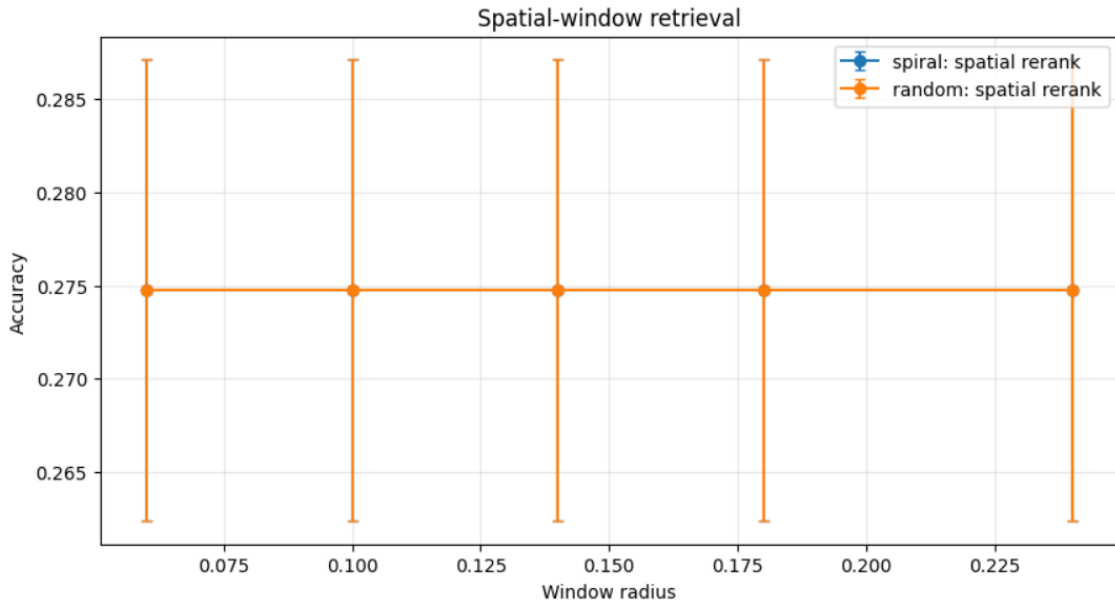


Figure 9: Run F spatial-window sweep. Spatial rerank accuracy stays flat across window radii, and spiral and random overlap almost exactly. This indicates that the window rule is not extracting additional value from placement geometry.

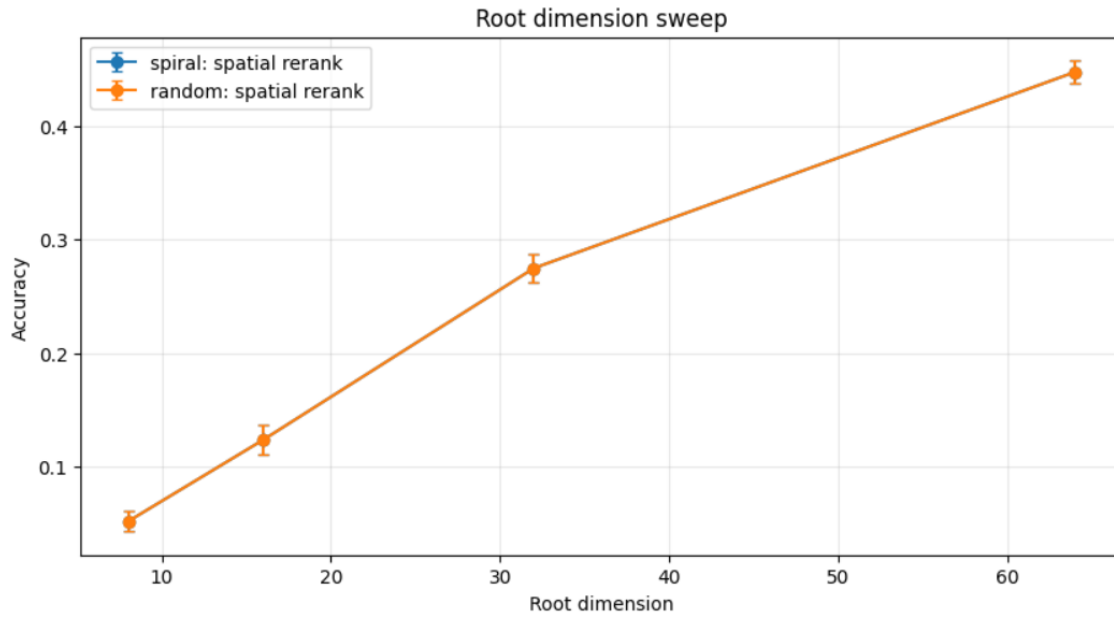


Figure 10: Run F root-dimension sweep. Performance still scales with root dimension, confirming that compression remains the dominant bottleneck even in the spatial-window variant.

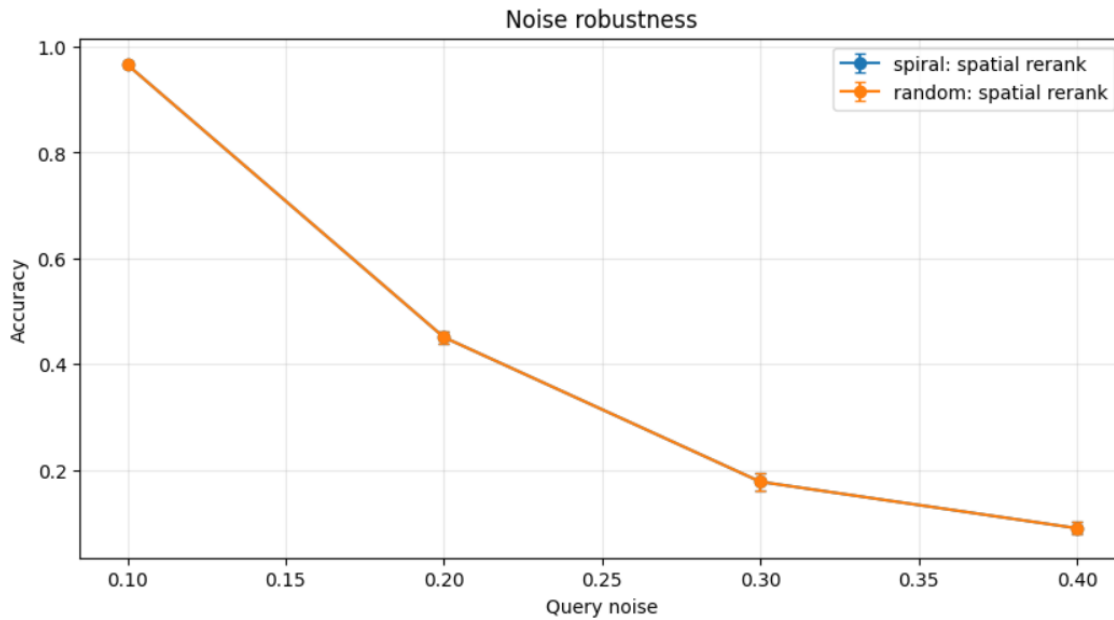


Figure 11: Run F query-noise sweep. Both placements degrade similarly as query noise rises, with no meaningful separation between spiral and random.

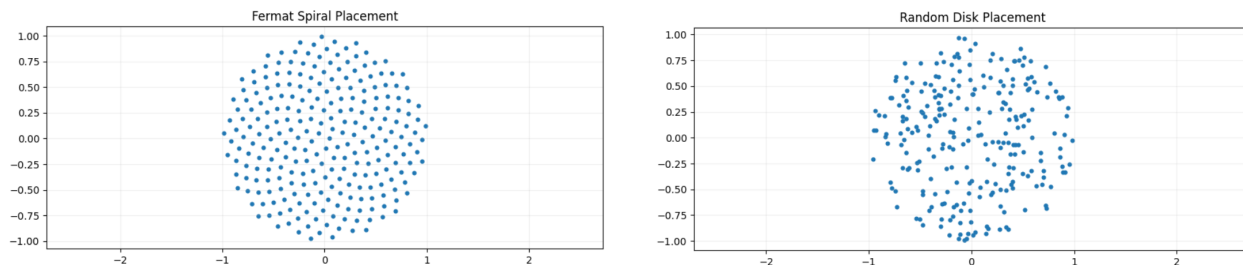


Figure 12: Run F placement geometry. The spiral still has cleaner spatial structure than random placement, but this structural difference does not improve spatial-window retrieval in the current passive setup.

A.7 Run G: Tiny Active Follicle Refinement (v5)

What was tested. This run asks whether adding a tiny active step inside a selected follicle helps recover a spiral advantage more reliably than the fully passive versions. The comparison includes three variants: passive top-1 retrieval, passive top- k pooling, and an *active_local* follicle that performs a small local refinement step after candidate selection.

How it was done. Memories remained clustered as in Runs E and F. For the active variant, each selected follicle pooled local neighborhood information and then applied a tiny near-identity transform followed by a Loose-Screw-style nonlinearity. The experiment compared the three variants at root dimension 32 and query noise 0.25, and then swept root dimension, query noise, and local radius for the active variant.

What happened. The main comparison shows a nuanced result. Passive top-1 remains the strongest item-level baseline, reaching 0.2747 for both placements. Passive top- k is clearly worse at 0.2409 for both placements. The new active-local variant lands in between: random reaches 0.2685 item accuracy while spiral reaches 0.2745, almost matching passive top-1 for spiral but not quite surpassing it. The same pattern appears in group accuracy, where spiral active-local reaches 0.3479 versus 0.3391 for random.

The sweeps show where the active step helps most. In the root-dimension sweep, spiral active-local is slightly better than random at all tested root dimensions, with the gap growing by root dimension 64 (0.4479 versus 0.4305). In the query-noise sweep, spiral also stays slightly ahead across all noise levels. The clearest separation appears in the local-radius sweep: as radius grows, spiral degrades more slowly than random. At radius 0.20, spiral still reaches 0.2534 item accuracy while random falls to 0.2122.

What we learned. This is the first run suggesting that a tiny active follicle may be more promising than additional passive locality tricks. The gain is still modest, and the active variant does not beat passive top-1 outright at the main operating point. However, unlike v4, it does recover a small and consistent spiral advantage, especially when the local neighborhood becomes larger or the root representation is stronger. A careful interpretation is that passive geometry alone is probably too weak, but a small active process inside each follicle may help the system exploit spatial structure more effectively.

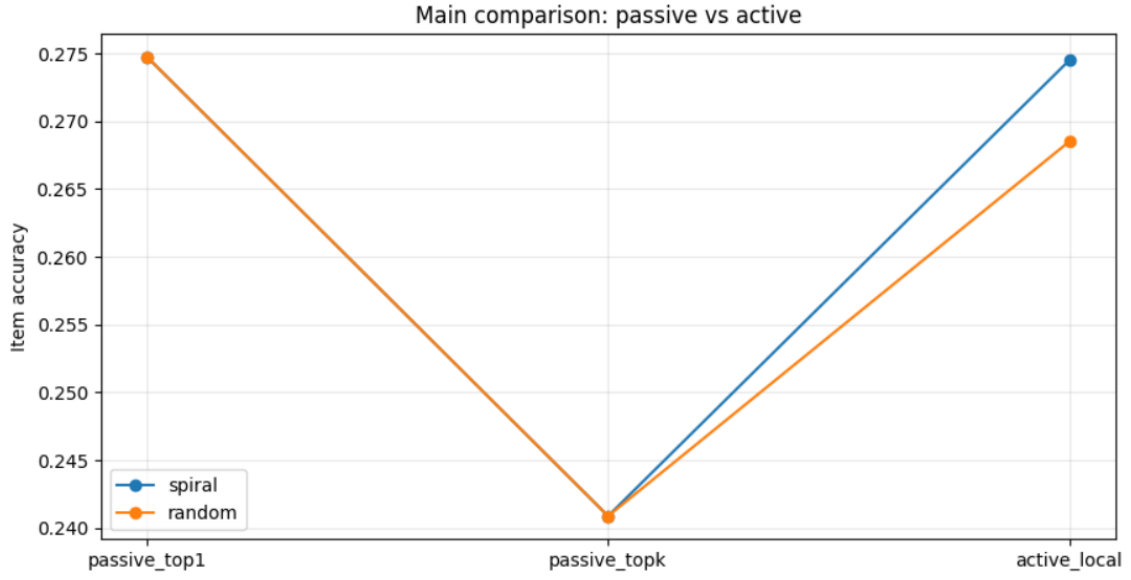


Figure 13: Run G main item-accuracy comparison. Passive top-1 remains the strongest baseline, passive top- k is worse, and the active-local variant brings spiral almost back to passive top-1 while leaving random lower.

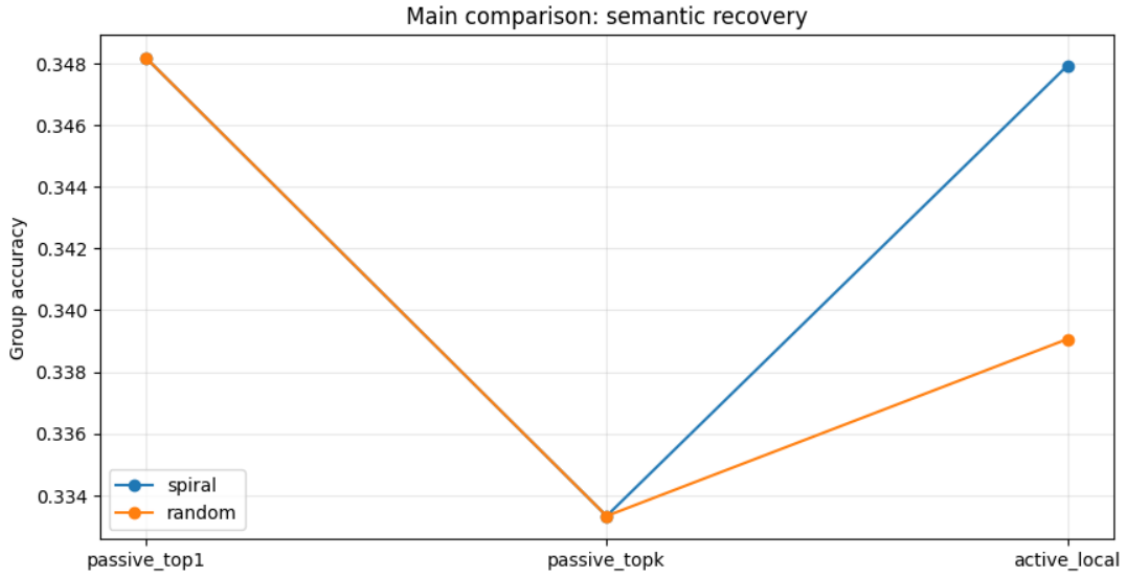


Figure 14: Run G semantic recovery comparison. Spiral active-local improves group-level recovery over random active-local, even though passive top-1 remains the overall strongest baseline.

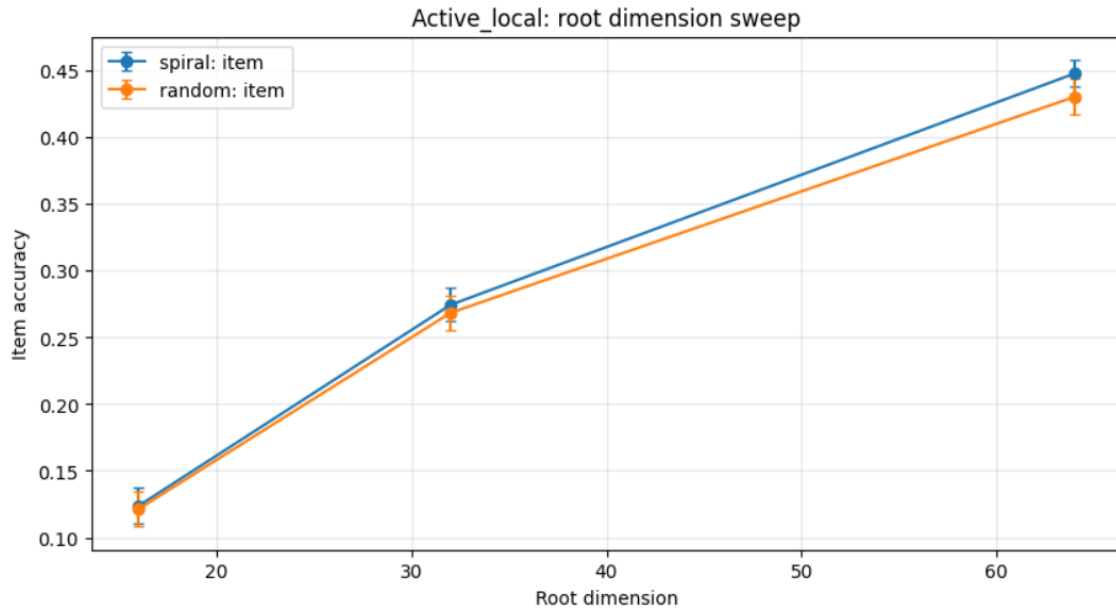


Figure 15: Run G active-local root-dimension sweep. Root quality still dominates, but spiral stays slightly above random as dimension increases.

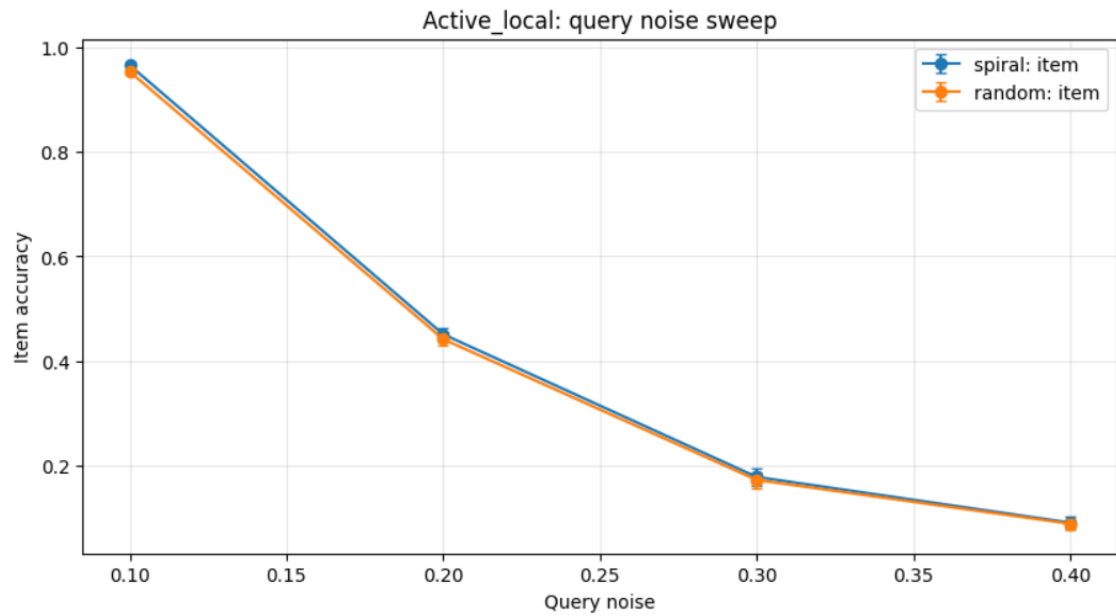


Figure 16: Run G active-local query-noise sweep. Both placements degrade with noise, but spiral remains marginally better across the tested range.

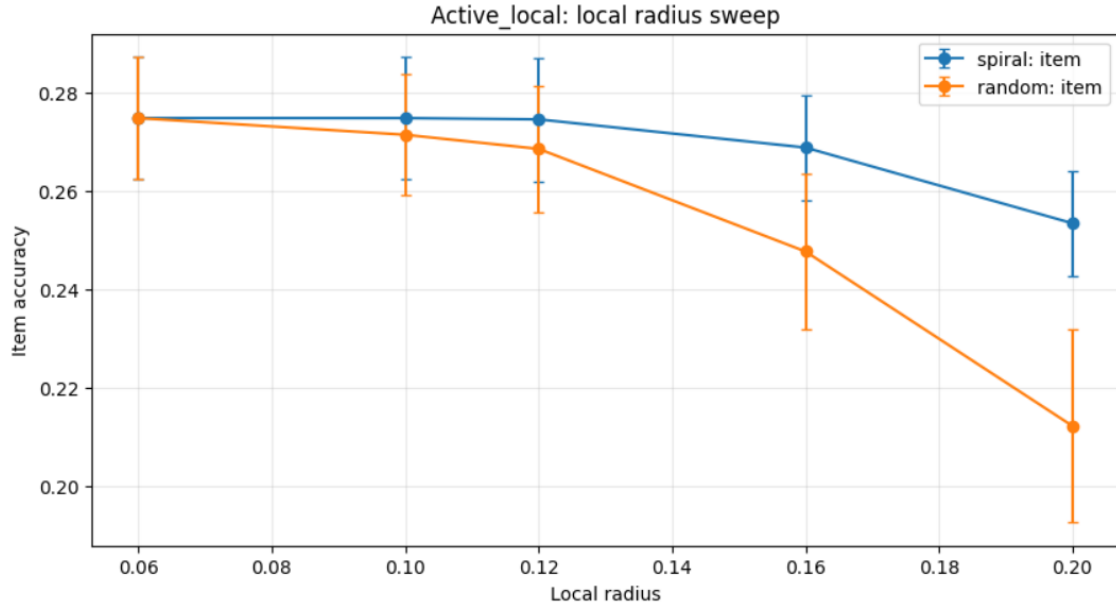


Figure 17: Run G active-local local-radius sweep. Spiral degrades more slowly than random as the active follicle is forced to use a larger neighborhood, which is the clearest v5 sign that activity helps exploit geometry.

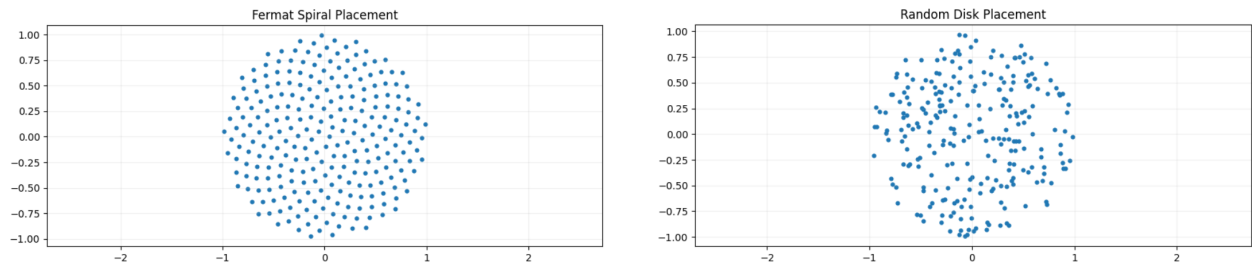


Figure 18: Run G placement geometry. The structural difference between spiral and random is unchanged, but in v5 the active follicle uses that structure a little more effectively than the passive variants did.

A.8 Run H: QDMN-Style Active Follicle (v6)

What was tested. This run asks whether a tiny QDMN-style processor placed inside each follicle can outperform the passive baselines. The idea is to go beyond passive lookup and let each selected follicle run a small internal route of transformations before decoding the answer.

How it was done. The comparison included three variants: passive top-1, passive top- k pooling, and a *qdmn_follicle* variant with a small internal route $F \rightarrow U \rightarrow B \rightarrow D \rightarrow H$. The follicle processor was trained on local context features built from the query root, candidate root, and neighborhood context. Root-dimension, local-radius, and query-noise sweeps were also run for the active follicle.

What happened. The main comparison reveals a clear failure mode. Passive top-1 still achieves item accuracy of about 0.2639, and passive top- k pooling reaches about 0.2413. In contrast, the QDMN follicle reaches exactly 0.0000 item accuracy for both spiral and random placement. However, it still shows nonzero group accuracy, around 0.1510 for random and 0.1337 for spiral. The root-dimension sweep shows the same pattern: passive performance improves with dimension, but the active follicle remains at zero exact-item accuracy. The query-noise sweep likewise shows that passive retrieval degrades normally with noise, whereas the active follicle stays at zero item accuracy throughout.

What we learned. This is an important negative result, but also an informative one. The QDMN-style follicle is not currently succeeding as an exact memory retriever. Instead, it appears to be learning a coarse semantic or group-level representation without preserving exact item identity. That interpretation is consistent with the nonzero group accuracy despite zero item accuracy. A likely reason is that the current training target uses embedding regression, which makes it easier to learn broad semantic structure than exact discrete identity. So v6 does not validate the active QDMN follicle as written. If this direction is continued, a better next step may be to train for group prediction first or switch the active follicle to an explicit exact-item classification objective.

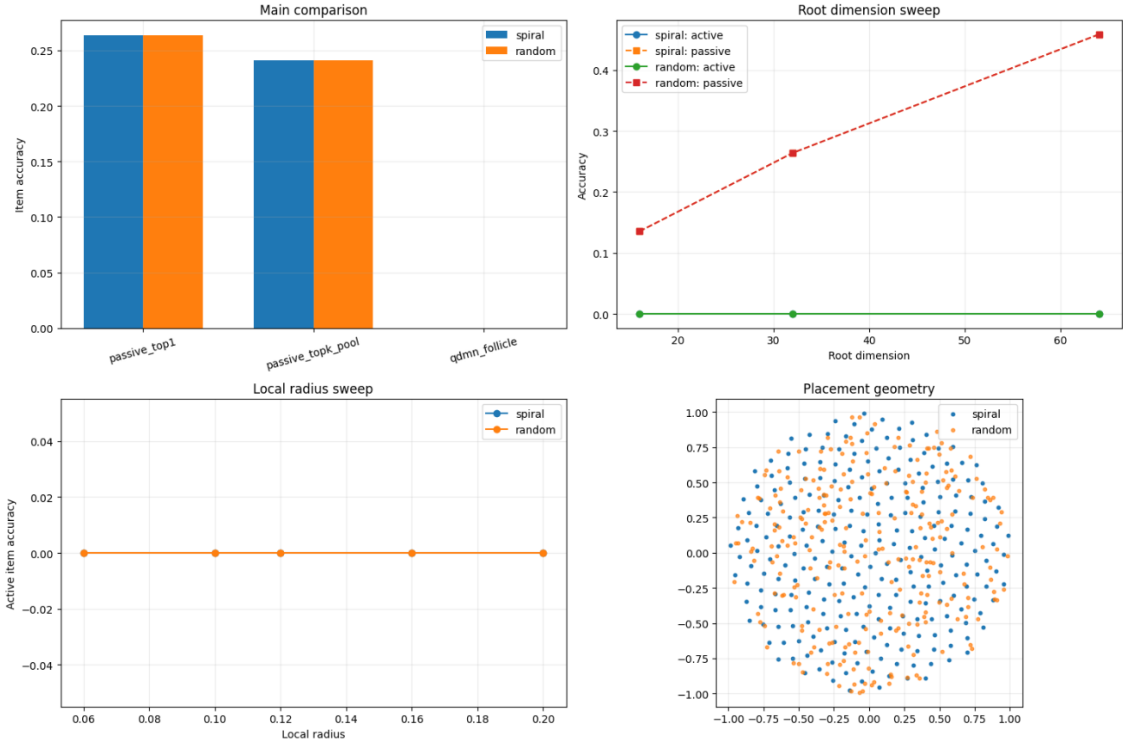


Figure 19: Run H combined summary. Passive baselines still work, but the QDMN follicle collapses to zero exact-item accuracy. The root-dimension and local-radius panels show that this failure persists even as passive retrieval remains healthy.

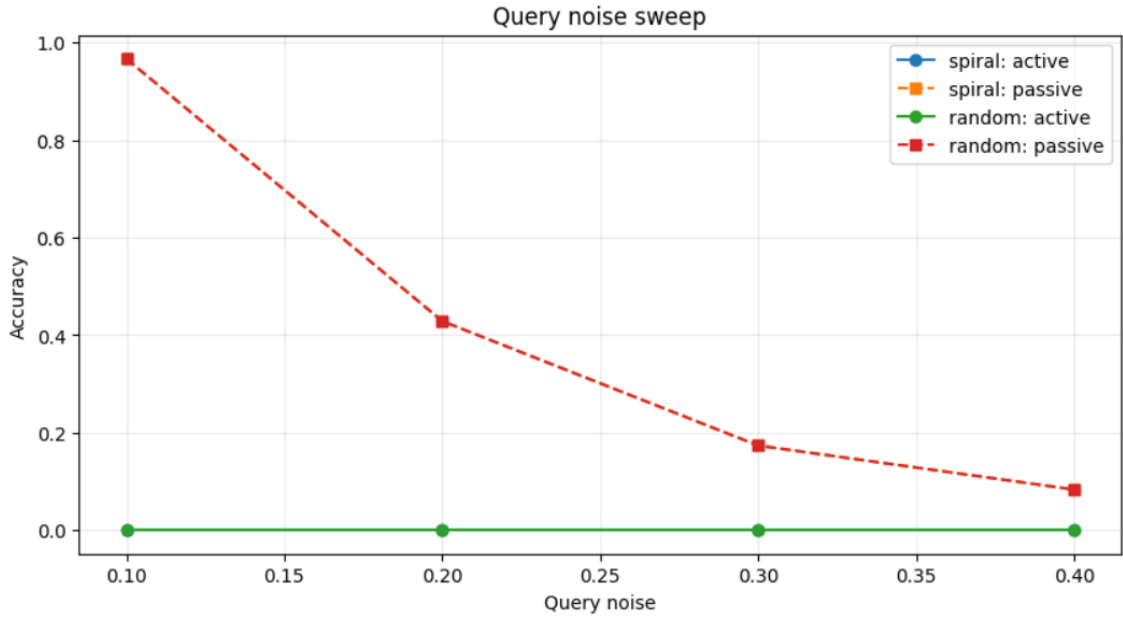


Figure 20: Run H query-noise sweep. Passive retrieval degrades with noise in the expected way, while the QDMN follicle remains at zero exact-item accuracy across all tested noise levels.

A.9 Run I: Cause-Pinpoint Diagnostic (Part 3)

What was tested. This run was designed as a diagnosis rather than a new architecture. The goal was to separate three possible failure sources in the hair-memory setup: root compression loss, noisy neighborhood aggregation, and spatial contamination from write leakage. Instead of introducing a more complicated follicle processor, the run compares several deliberately simple retrieval rules to see which failure source is currently dominant.

How it was done. Five retrieval modes were compared: passive top-1, passive top- k pooling, clean-jury best-1, context-soup pooling, and selective routing. The clean-jury variant chooses a single best local neighbor rather than pooling many neighbors. The context-soup variant deliberately averages a local neighborhood, making it a direct test of whether local pooling introduces noise. The selective-route variant gates between the global candidate and the best local neighbor. Multi-seed sweeps were then run over root dimension, query noise, write leak, and local radius for both spiral and random layouts. In addition to item and group accuracy, the run also recorded root hit rates, local coverage of the true item, neighborhood size, gate values, and basic geometry statistics.

What happened. The main comparison is striking because it is so stable. At the default setting, passive top-1 reaches item accuracy 0.2773 for spiral and 0.2826 for random. Clean-jury best-1 and selective routing are effectively identical to those values, while context-soup pooling is worse at 0.2663 for spiral and 0.2578 for random, and passive top- k pooling is worse still at 0.2461 for spiral and 0.2428 for random. The root-dimension sweep shows the same ordering at every tested dimension: performance rises from about 0.04 at dimension 8 to about 0.44 at dimension 64, and the clean-jury and selective-route variants simply ride on top of passive top-1 rather than improving it. The query-noise sweep shows another strong pattern: all methods are near-perfect at noise 0.10, collapse to roughly 0.46 by noise 0.20, and fall below 0.10 by noise 0.40, again with only small differences among the top-1-style rules.

The write-leak sweep is comparatively flat. Over leak values from 0.00 to 0.20, passive top-1 only drifts from 0.2852 to 0.2760 for spiral and from 0.2852 to 0.2806 for random. That decline is real, but it is much smaller than the swings caused by root dimension or query noise. The local-radius diagnostic adds one more useful clue. Local coverage of the true item rises as radius expands, reaching about 0.6120 for spiral and 0.6393 for random at radius 0.20, yet clean-jury accuracy stays almost perfectly flat near 0.277 to 0.283. So giving the jury access to more of the right local region does not translate into better retrieval under the current decoding rule.

What we learned. This run narrows the failure point substantially. The most damaging operation is not moderate write leakage, and it is not the mere absence of local candidates. If either of those were the primary problem, then clean-jury or selective routing should have produced clear gains once local coverage increased. Instead, the near-perfect tie between passive top-1, clean-jury, and selective-route suggests that the global top-1 candidate is already doing almost all the useful work. Meanwhile, both context-soup pooling and passive top- k pooling consistently hurt accuracy, which means that averaging several shafts or several neighbors is currently more harmful than helpful. The cleanest interpretation is therefore that root compression and pooled decoding remain the main bottlenecks, while neighborhood aggregation mainly adds noise unless a much sharper routing rule is introduced. In short: the jury is not blind, but it is not yet adding useful evidence.

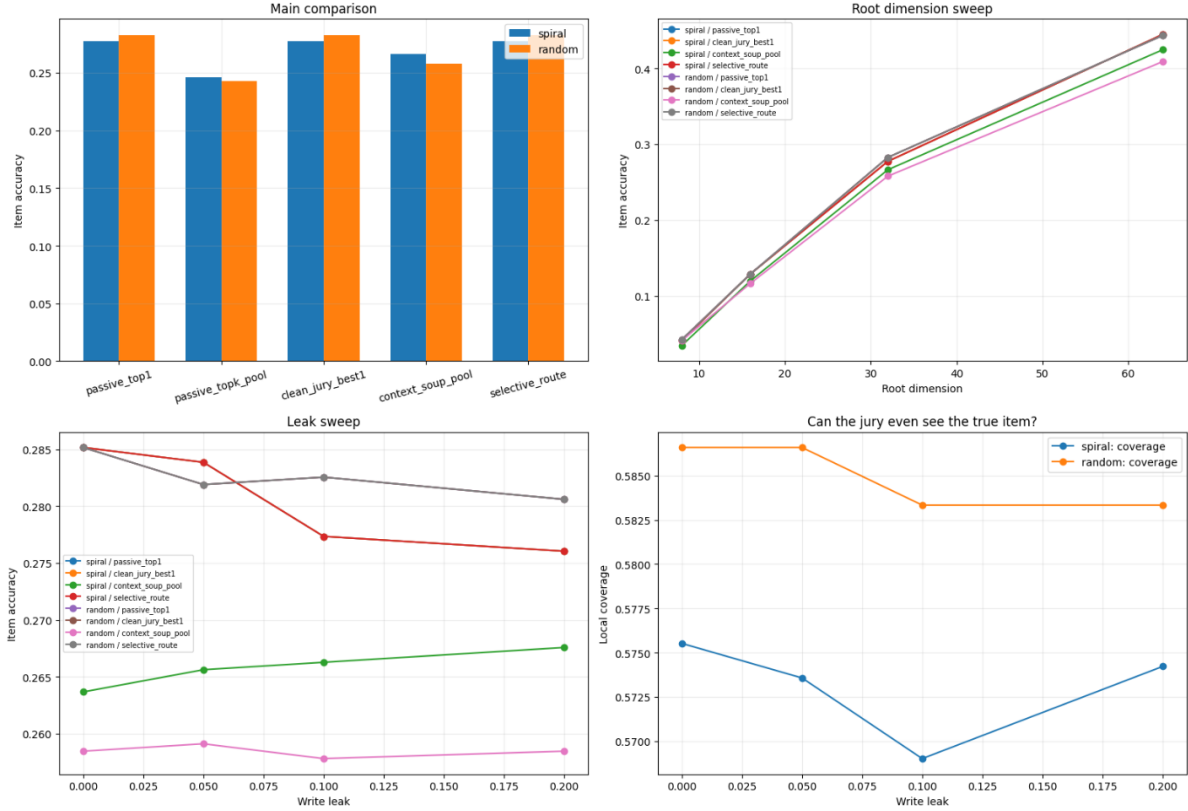


Figure 21: Run I cause-pinpoint overview. The main comparison, root-dimension sweep, leak sweep, and local-coverage panel all point to the same conclusion: passive top-1, clean-jury best-1, and selective routing behave almost identically, while pooled variants are worse. Root dimension and query noise matter much more than moderate write leak.

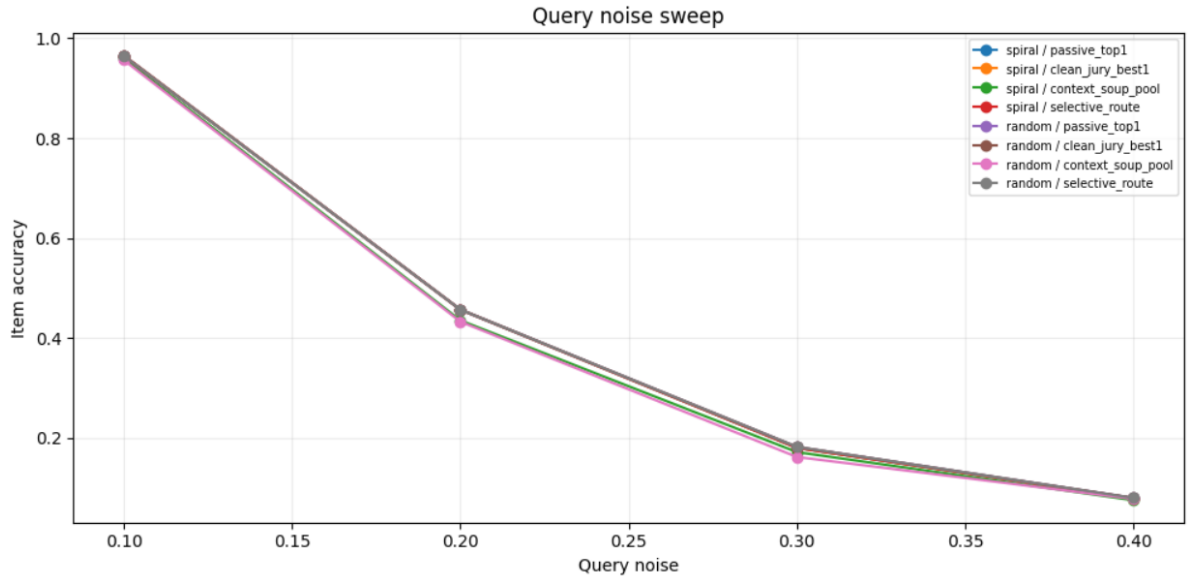


Figure 22: Run I query-noise sweep. All retrieval rules degrade rapidly as query noise increases, and the top-1-style methods remain nearly tied throughout. This reinforces the conclusion that noisy queries and compressed roots are stronger bottlenecks than local jury design in the current setup.

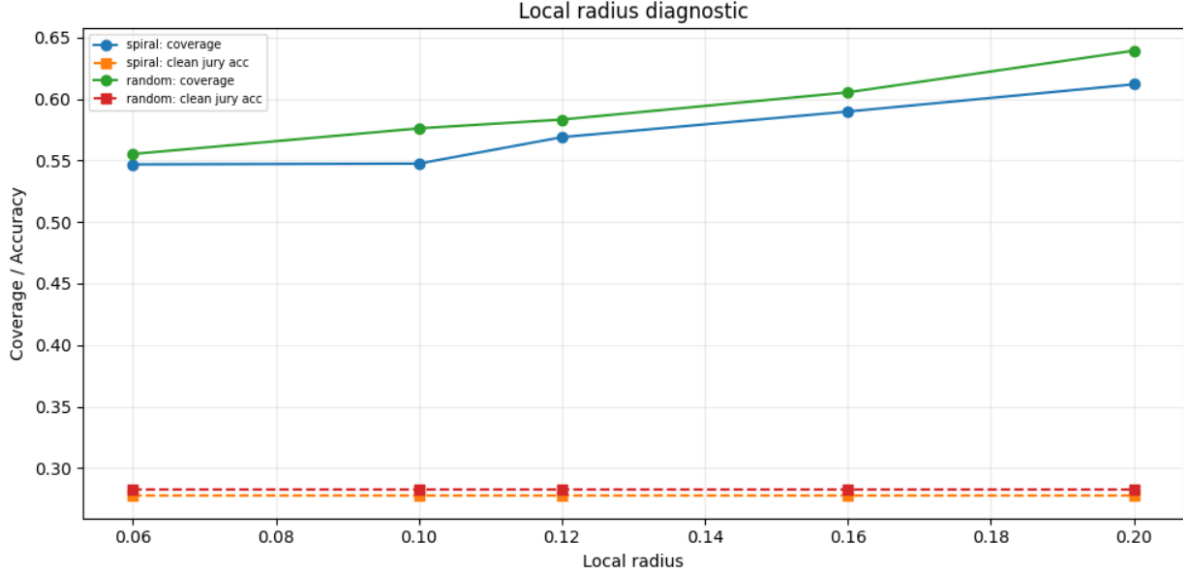


Figure 23: Run I local-radius diagnostic. Enlarging the local radius clearly increases coverage of the true item, but clean-jury accuracy stays almost flat. This suggests that simply seeing more of the neighborhood is not enough; the current system still lacks a mechanism that converts extra local access into better retrieval.

A.10 Run J: Control-Oracles for Root vs Neighbor Failure (Part 4)

What was tested. This run extends the previous cause-pinpoint study by introducing explicit oracle controls. The goal is to separate two questions that the earlier diagnostic could only hint at: how much performance is lost because the root stage chooses the wrong candidate, and how much is lost because the local routing stage fails to exploit the candidates that are already available. The oracle methods are not deployable models; they are upper bounds used to show where the recoverable headroom is located.

How it was done. The run compares six modes: passive top-1, passive top- k pooling, local-jury best-1, selective routing, a *candidate oracle*, and a *local oracle*. The candidate oracle assumes that, among the global top- k candidates, the system can pick whichever one is best for the true target. The local oracle goes one step further and assumes that, across the union of local neighborhoods around the top- k candidates, the system can identify the best local item for the target. Multi-seed sweeps were then run over root dimension, query noise, local radius, and write leak for both spiral and random placement, while keeping the same clustered-memory setup as the previous diagnostic.

What happened. The main comparison reveals a very clean three-way separation. At the default setting, passive top-1 reaches 0.2625 item accuracy for spiral and 0.2451 for random. The candidate oracle then jumps to 0.5240 for spiral and 0.5174 for random, while the local oracle rises further to 0.5445 and 0.5495. That means the largest recoverable gap is already present at the root stage, but there is also a second smaller gap available at the neighborhood stage once the right local item is identified. Selective routing, however, remains almost identical to passive top-1, and local-jury best-1 is slightly worse for random and essentially tied for spiral. So the practical local methods are recovering almost none of the signal that the oracle controls reveal.

This also clarifies what is failing inside the neighborhood. The local set is often informative: by construction the local oracle can exploit it, and local coverage remains substantial. But the practical aggregation rules behave destructively. In effect, the neighborhood contains a mixture of one useful signal and many distractors, and naive pooling or weak jury rules wash the signal out instead of isolating it. So the problem is not that neighbors are useless; it is that the current aggregation and

routing rules are not discriminative enough.

The sweeps reinforce the same interpretation. In the root-dimension sweep, passive top-1 rises from roughly 0.05 at dimension 8 to about 0.42 at dimension 64, but the two oracle curves stay far above it throughout, reaching about 0.71–0.73 by dimension 64. That means better roots help, but even when the root representation improves there is still large unrealized value in choosing better candidates or better local items. In the local-radius sweep, the local oracle improves substantially as radius expands, especially for random placement, while the practical local methods often stay flat or even worsen. This indicates that additional local access does contain useful information, but the current jury and gating rules are not extracting it. The write-leak sweep shows that all methods decline with contamination, yet the oracle gap remains large across the entire leak range, so the main limitation is not erased by simply reducing or increasing leakage.

What we learned. This run gives the clearest causal decomposition in the paper so far. Root compression is the dominant bottleneck, because the jump from passive top-1 to the candidate oracle is large. But neighbor selection also contains real recoverable signal, because the local oracle still sits above the candidate oracle. The correct summary is therefore more precise than saying that neighbor pooling is simply harmful. What the experiment actually shows is that *naive* neighbor pooling is harmful, while the neighborhood itself contains useful signal that a better selector could recover.

This also explains the selective-router failure. The gate values stay near one, yet performance remains near passive top-1. That is strong evidence that the router is not distinguishing signal from noise; it is effectively passing control to a local stage that still aggregates neighbors in a weak way. Geometry remains a second-order effect in comparison. Spiral slightly improves some practical curves, but the oracle gap is present for both placements, which means that the primary failure is still selection rather than the mere existence of spatial structure. A balanced reading is therefore that both stages matter: better roots raise the ceiling, and better routing is needed to capture the ceiling that is already available inside the current candidate and neighborhood sets.

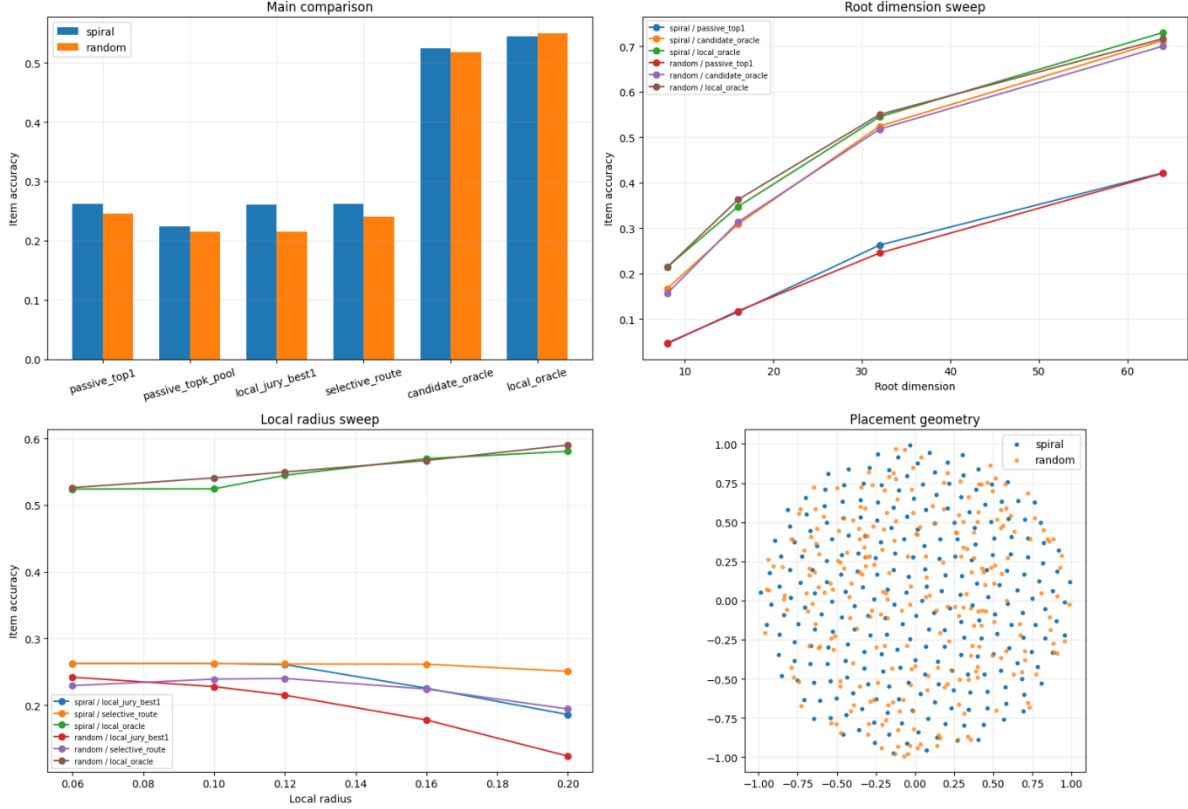


Figure 24: Run J oracle-control overview. The main comparison, root-dimension sweep, local-radius sweep, and placement geometry panel show a consistent pattern: the oracle methods sit far above the practical methods, while selective routing remains close to passive top-1. This indicates that the current system leaves substantial recoverable signal unused inside the existing top- k and neighborhood sets.

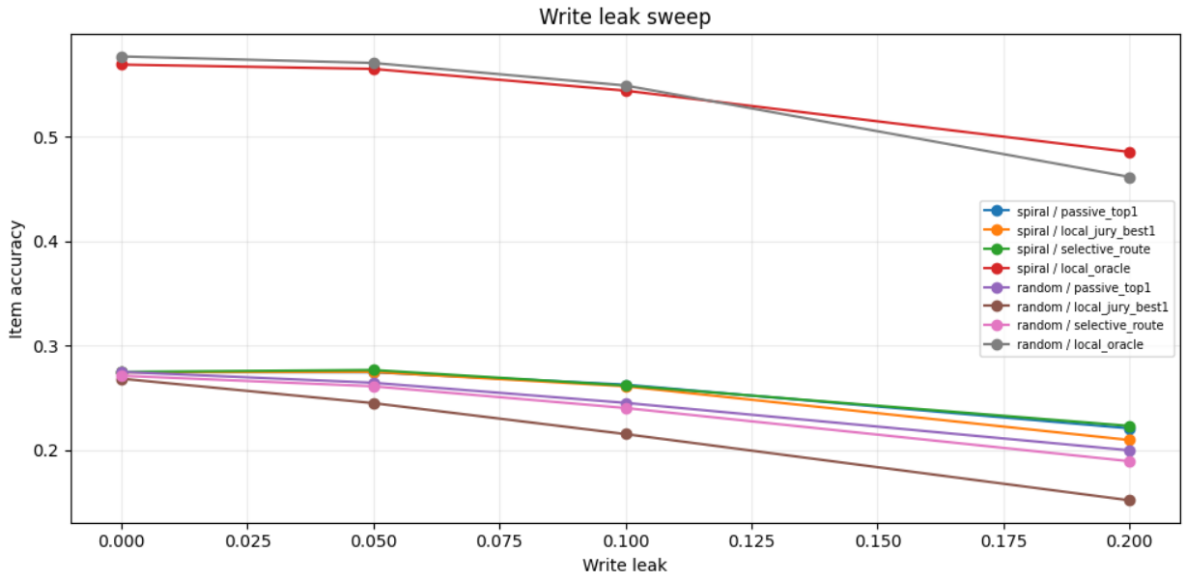


Figure 25: Run J write-leak sweep. Performance declines as contamination increases, but the oracle gap remains large across the whole leak range. This suggests that write leakage is not the only problem; better candidate selection and local routing could still recover much more of the available signal.

A.11 Run K: Neighbor Aggregation Comparison (Part 5)

What was tested. This run asks a narrower question than the earlier oracle study: given the same memory field, which simple local aggregation rule works best once a top root candidate has already been selected? The goal is not to propose a new architecture, but to determine whether local averaging, local top-1 picking, or temperature-controlled local softmax pooling can turn neighborhood information into a real retrieval gain.

How it was done. The comparison included five modes: passive top-1, passive top- k pooling, local average pooling, local top-1 pick, and local softmax pooling. The local methods all used the best global candidate as an anchor, then inspected the spatial neighborhood around that anchor within a chosen radius. The average rule pooled the local shafts uniformly, the top-1 rule chose the highest-scoring local item, and the softmax rule used temperature-controlled weighting over local root scores. Multi-seed sweeps were then run over root dimension, query noise, local radius, write leak, and softmax temperature for both spiral and random placement.

What happened. The answer is cleaner than expected. At the main operating point, passive top-1 reaches 0.2674 item accuracy for spiral and 0.2478 for random. Local top-1 pick is effectively identical to those values. Local softmax pooling is the strongest local variant, but only by a hair: it reaches 0.2487 for random and remains essentially tied with passive top-1 for spiral. In contrast, local average pooling collapses badly, falling to 0.0929 for spiral and 0.0768 for random. Passive top- k pooling is also clearly worse than passive top-1.

The sweeps reinforce that pattern rather than weakening it. Across root dimension and query noise, local top-1 and local softmax pooling continue to track passive top-1 almost exactly, while local average pooling remains far worse. The local-radius sweep is especially informative. For random placement, local average pooling gets steadily worse as radius grows, and for spiral it stays harmless only at very tiny radii where the neighborhood is effectively just the anchor itself. Once the radius widens enough to include a real neighborhood, average pooling degrades sharply. The softmax-temperature sweep shows the same story in another form: performance is best at low temperatures and worsens as the softmax becomes flatter. The best temperature is 0.15 for random placement and 0.05 for spiral placement, but even those best settings improve only marginally over passive top-1. That means the winning version of local pooling is the one that behaves most like a nearly hard selection rule rather than a true average.

What we learned. This run narrows the next step substantially. We still do not have a strong neighborhood solution. The best local method is not a rich neighborhood combiner; it is a very conservative softmax that stays extremely close to the anchor. That means the root candidate already carries most of the useful signal, while the neighborhood contributes only a small amount of extra information under the current setup. The practical implication is important: the problem is not solved by adding more pooling. Averaging neighbors is actively destructive, and even top-1 neighbor selection is mostly neutral. The most honest conclusion is therefore that present neighborhood methods are neutral or harmful except for very light smoothing.

That is still a useful negative result. It suggests that the next experiment should not ask how to blend more neighbors, but how to introduce a sharper decision rule that only overrides the anchor when a local alternative is clearly superior. In other words, the next gain is more likely to come from hard selection or confidence-based routing than from broader local averaging.

A.12 Run L: Root Fixes, Query Reranking, and Multi-Root Union (Part 6)

What was tested. This run compares three solution families directly: stronger root search, query-based local reranking, and multi-root union with refinement or gating. The goal is to determine which part of the retrieval pipeline now matters most once naive neighbor averaging has been removed.

How it was done. Seven modes were evaluated: passive top-1, passive top- k pooling, single-root rerank, single-root refine-and-rerank, multi-root rerank, multi-root refine-and-gate, and an oracle union control. The single-root modes start from one projected root search and then rescore a candidate neighborhood by query-key similarity. The multi-root modes instead use several independent root projections, take the union of the resulting candidate sets, and then make a hard final choice inside that richer union. Multi-seed sweeps were run over root dimension, number of projections, root shortlist size, query noise, and placement geometry for both spiral and random layouts.

What happened. The result is much cleaner than in the earlier local-pooling runs. At the main operating point, passive top-1 reaches only 0.2598 item accuracy, while single-root rerank roughly doubles that to 0.5189 for random placement and 0.5150 for spiral. The larger jump comes from multi-root union: multi-root rerank reaches 0.7656 for random and 0.7689 for spiral, while the oracle union reaches 0.8971 for both. By contrast, the extra refine-and-gate step is not helpful. Multi-root refine-and-gate falls to about 0.48, well below plain multi-root rerank, and passive top- k pooling remains slightly worse than passive top-1.

The sweeps show the same pattern from several angles. As root dimension increases from 8 to 64, every method improves, but multi-root rerank stays the strongest practical method throughout. Increasing the number of projections dramatically expands candidate coverage, from about 0.56 with one projection to about 0.97 with eight, and multi-root rerank tracks that gain closely. Increasing the shortlist size from 1 to 10 produces the same effect: the candidate union becomes much more likely to contain the correct item, and multi-root rerank rises accordingly. Query noise then reveals the remaining limitation. At low noise, nearly all methods saturate, but at moderate and high noise the gap between multi-root rerank and oracle union stays substantial. That means the candidate set usually contains the answer, yet the practical scorer still fails to select it consistently.

Figure 26 collects the main fixed-point comparison, the root-dimension sweep, the projection-count sweep, and a placement-geometry panel. The visual summary makes the ranking especially clear: passive variants remain weak, single-root reranking gives a substantial lift, multi-root reranking gives the largest practical gain, and the oracle remains above all practical methods. Figure 27 then shows the query-noise sweep, where the same ordering persists as the task becomes harder. Multi-root rerank is robustly above the single-root methods, but it still trails the oracle at moderate and high noise, which again points to imperfect final discrimination rather than missing coverage.

Placement effects remain secondary. Spiral continues to produce better nearest-neighbor spacing and slightly lower interference, and it is marginally ahead of random in some multi-root rerank rows. However, those differences are tiny compared with the gains from reranking and richer candidate unions. The dominant effect in this run is retrieval design rather than geometry.

What we learned. This is the clearest positive result in the project so far. Query-based reranking works: the query contains useful information that earlier pooling rules were discarding. Stronger root search works too: a single fragile root is no longer the ceiling once several projections are allowed to contribute candidates. The most important diagnosis is therefore that the current system is no longer failing mainly because it cannot *reach* the right memory. With multi-root union, candidate coverage is already very high. The main remaining error is *discrimination within*

the candidate union: the final scorer is still too weak to identify the correct item reliably once the correct item is present.

The negative result is equally useful. Refinement and gating, at least in the present form, do not improve the decision boundary; they make it worse. So the next step should not be another routing layer or another pooling rule. The strongest next experiment is to keep multi-root rerank as the core retrieval mechanism and improve only the final scorer inside the union, for example with a sharper query–candidate comparison or a more selective hard decision rule.

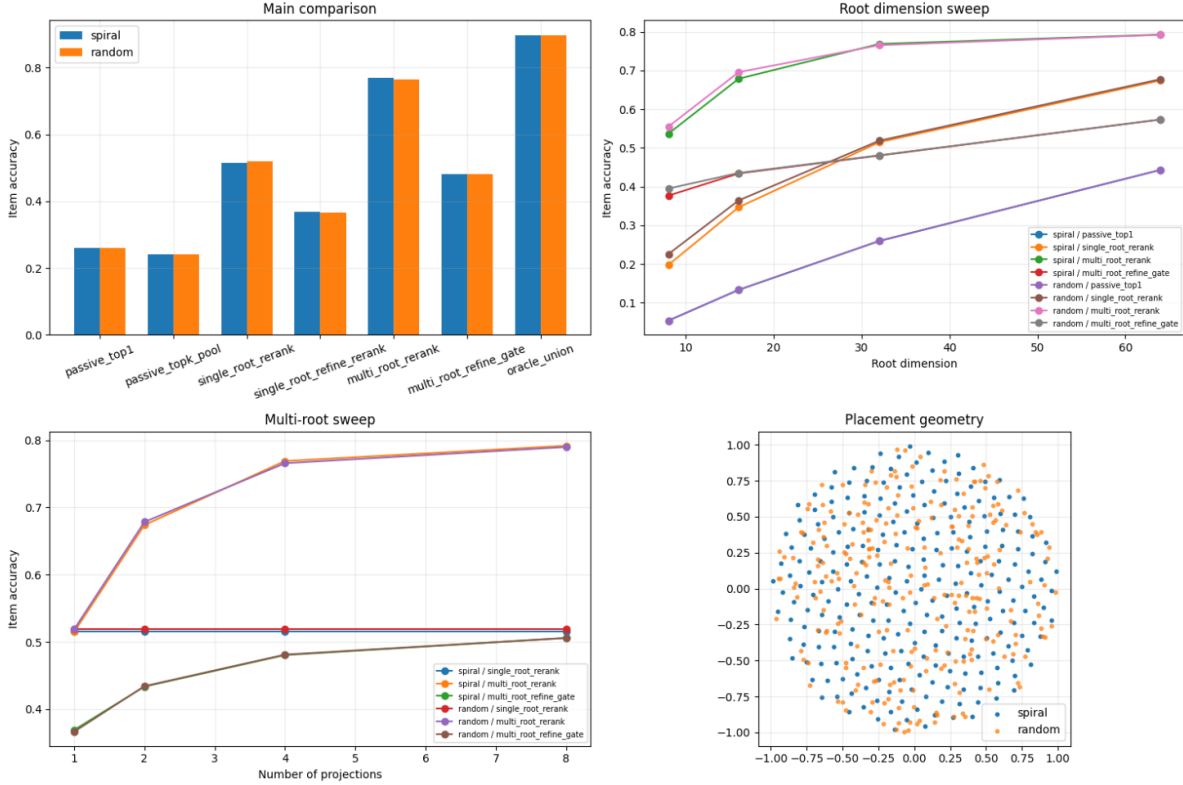


Figure 26: Run L overview. The main comparison, root-dimension sweep, projection-count sweep, and placement-geometry panel all tell the same story: query-based reranking helps substantially, multi-root union helps much more, and the remaining gap to the oracle is primarily a scoring problem inside the candidate union rather than a coverage problem. The geometry panel again confirms that spiral placement is more regular, but the dominant gains come from retrieval design rather than placement alone.

A.13 Run M: Union, Rerank, and Scale Diagnostic (Part 7)

What was tested. This run asks three direct questions. First, can the current hair-memory retrieval pipeline beat a simple flat nearest-neighbor baseline that searches all keys directly? Second, when hair memory falls short, is the main missing piece candidate discovery or reranking quality? Third, do those relationships change as the number of stored groups increases?

How it was done. The comparison was deliberately compact. Five modes were evaluated: flat key nearest-neighbor, passive root top-1, single-root rerank, multi-root rerank, and an oracle union. The oracle union uses exactly the same multi-root candidate set as the practical system, but returns the true item whenever it is present in that set, so it cleanly upper-bounds what the current candidate discovery stage makes available. The sweeps then varied the root dimension, the root shortlist size, the number of projections, the query noise, and the total number of groups for both spiral and random placement.

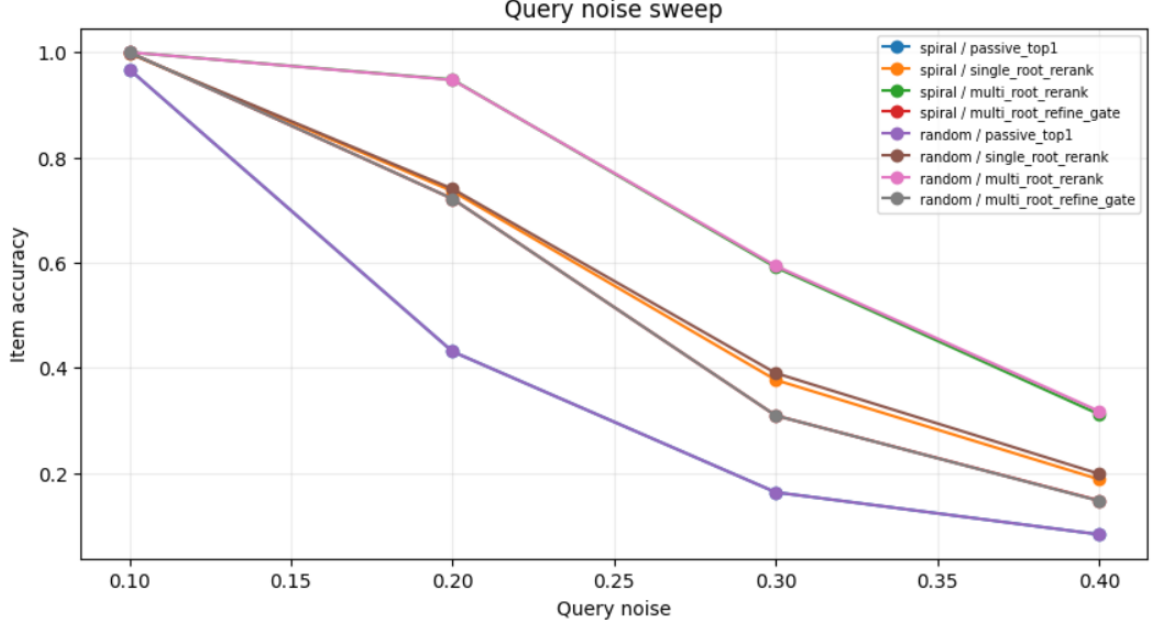


Figure 27: Run L query-noise sweep. All methods degrade as query noise increases, but multi-root rerank remains consistently stronger than single-root rerank and passive top-1 across the full range. The persistent gap between multi-root rerank and the oracle at moderate and high noise indicates that the correct item is often already present in the union, yet the practical scorer still fails to choose it reliably.

What happened. The main result is more sobering than Part 6. At the main operating point, the flat key baseline reaches 0.8021 item accuracy for both placements, while multi-root rerank reaches 0.7734 for spiral and 0.7765 for random. So the present hair mechanism still does *not* beat a strong flat baseline on exact retrieval. At the same time, the oracle union reaches 0.9032 for spiral and 0.9097 for random. That split is the key diagnostic. The current candidate union is often rich enough to contain the correct item, but the practical reranker does not score the items inside that union well enough to convert that opportunity into a win over flat search.

The detailed sweeps reinforce that reading. Increasing root dimension, shortlist size, and especially the number of projections all improve multi-root rerank substantially by enlarging candidate coverage. But even at the best practical settings shown here, multi-root rerank remains below flat key nearest-neighbor, while oracle union stays comfortably above it. Query noise shows the same pattern in another form. At low noise, both flat search and multi-root rerank are nearly perfect. At moderate and high noise, multi-root rerank remains much better than the passive root-only methods, yet it still does not close the remaining gap to either the oracle or the flat baseline. The scale sweep is especially informative. As memory size grows from 32 to 128 groups, both flat search and multi-root rerank degrade, but the relative ranking does not flip: flat search remains better than practical hair retrieval, while oracle union remains better than both.

The run also contains one useful negative result. With the chosen best setting, single-root rerank and passive root top-1 are numerically identical. In this implementation, the extra reranking support term contributes no practical gain at the selected β value, so the real benefit still comes from multi-root candidate expansion rather than from the current scoring blend itself.

Figure 28 shows the main operating-point bars for spiral and random placement. The figure makes the central comparison easy to read: flat key nearest-neighbor is stronger than the current practical hair method, but the oracle union is stronger than both. That is exactly the signature expected when candidate discovery is good enough in principle, yet final reranking remains too weak in

practice.

What we learned. Part 7 sharpens the diagnosis from Part 6 instead of overturning it. Multi-root union is still the right retrieval skeleton, because it increases coverage dramatically and produces a strong oracle ceiling. But the present hair system is not yet competitive with a strong flat baseline. The bottleneck is no longer merely whether the system can discover a useful candidate pool; it is whether the final scorer can exploit that pool well enough to beat direct key search. In that sense, this run strengthens the case for focusing next on a better within-union scorer rather than on more pooling, more geometry tweaks, or small adjustments to the current rerank blend.

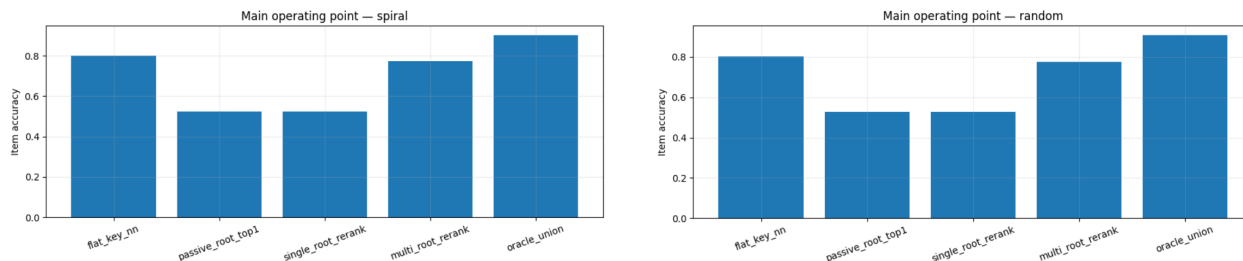


Figure 28: Run M main operating-point comparison for spiral and random placement. In both layouts, the practical multi-root reranker improves strongly over the passive root-only path, but it still remains below flat key nearest-neighbor. The oracle union remains above both, showing that the discovered candidate set is often strong enough in principle and that the remaining practical gap is mainly a reranking or scoring problem inside that union.

A.14 Run N: Gap Closer — Candidate Discovery vs. Candidate Judgment (Part 8)

What was tested. This run asks the sharpest version of the current question: once a strong multi-root candidate pool has already been found, is the remaining error mostly in candidate discovery or in candidate judgment? It also asks whether a slightly richer hand-built score or a simple learned judge can close more of the remaining gap.

How it was done. The candidate discovery stage was held fixed at the strongest multi-root setup from the earlier runs. Seven modes were then compared: flat key nearest-neighbor over all items, passive root top-1, single-root rerank, multi-root rerank, a mixed multi-root rerank that combines flat, root, and local signals, a learned linear judge trained on a held-out split by group, and an oracle union that serves as the upper bound for the fixed candidate pool. Additional sweeps varied root dimension, shortlist size, number of projections, query noise, and memory scale.

What happened. The main comparison is informative precisely because it is somewhat disappointing. At the operating point shown in Figure 29, flat key nearest-neighbor reaches 0.7917 item accuracy for both placements. Multi-root rerank and the mixed heuristic rerank are lower, at about 0.76, while the learned linear judge is lower still at about 0.68. Meanwhile, the oracle union remains much higher: 0.8958 for spiral and 0.9097 for random. So the strong candidate pool still contains enough signal to beat the flat baseline in principle, but none of the practical judges here can exploit that opportunity.

The sweeps sharpen that diagnosis. As root dimension, shortlist size, and projection count increase, candidate coverage rises strongly and all union-based methods improve. But the ranking among the practical methods changes little: the mixed heuristic score is at best only a negligible change from plain multi-root rerank, and the learned linear judge remains below the heuristic multi-root methods across the main settings. Query noise tells the same story. At low noise, every good method saturates. At moderate and high noise, the practical judges all fall well below the oracle,

which again indicates that the current issue is not simply whether the right candidate is visible. The scale sweep adds one more useful caution. At 32 groups under spiral placement, the learned linear judge collapses abnormally, even though the heuristic multi-root methods remain strong. That suggests the current learned judge is not only weak but also unstable under some train/test splits and geometries.

Figure 29 summarizes the main operating-point comparison for both placements. The bar ordering makes the central lesson easy to see: oracle union is best, flat key nearest-neighbor is the strongest practical baseline, the multi-root heuristic judges are meaningfully above the passive root-only path, and the learned linear judge does not close the gap.

What we learned. Part 8 strengthens the story from Parts 6 and 7 rather than changing it. Candidate discovery is no longer the only serious limitation, because the oracle union remains very strong. But improving the judge is harder than simply adding a few extra handcrafted features or fitting a shallow linear classifier on top of the current feature set. In other words, the diagnosis is now more precise: the remaining gap really is in candidate judgment, yet the present judgment mechanisms are not expressive or stable enough to recover the oracle-accessible signal. The next step should therefore be a stronger scorer, but probably not just a slightly richer linearized one.

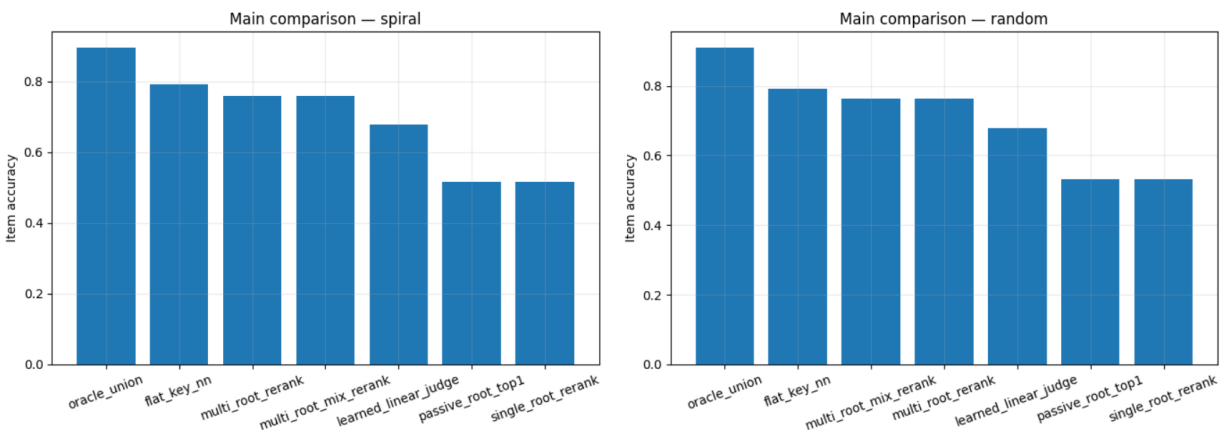


Figure 29: Run N main comparison for spiral and random placement. The oracle union remains well above all practical methods, while flat key nearest-neighbor remains the strongest practical baseline. Multi-root heuristic reranking stays clearly above passive root-only retrieval, but neither the mixed heuristic judge nor the learned linear judge closes the remaining gap to flat search. This indicates that the candidate pool is strong enough in principle and that the unresolved problem lies in candidate judgment rather than simple candidate discovery.

A.15 Run O: Loose-Screw and Selective Guard on a Fixed Multi-Root Pool (Part 9)

What was tested. This run keeps candidate discovery fixed and asks whether two further judgment ideas help: a small Loose-Screw style judge with a contractive refinement block, and a selective guard that falls back to a heuristic judge when the learned judge appears uncertain. The central question is whether these additions can close more of the gap between practical retrieval and the oracle candidate pool.

How it was done. The same strong multi-root candidate pool from the previous parts was reused. The comparison included flat key nearest-neighbor, passive root top-1, single-root rerank, multi-root rerank, mixed multi-root rerank, a learned linear judge, a Loose-Screw judge, a guarded linear judge, a guarded Loose-Screw judge, and the oracle union. The guarded variants trust the learned judge only when its own top-margin is large enough; otherwise they fall back to the mixed heuristic

scorer. Sweeps then varied root dimension, root shortlist size, projection count, query noise, and scale.

What happened. The main result is modest but useful. The oracle union again remains the ceiling, reaching 0.9063 for spiral and 0.9080 for random at the main operating point. Flat key nearest-neighbor remains the strongest practical baseline at 0.7847. The best practical hair variants are now slightly improved relative to Part 8: the mixed multi-root reranker reaches 0.7622, and the guarded linear judge reaches 0.7604 for both placements. The guarded Loose-Screw judge improves over the unguarded Loose-Screw judge, but it still trails the guarded linear judge and the mixed heuristic reranker. So the new guard helps, but only a little.

The sweeps reinforce that interpretation. Across root dimension, shortlist size, and projection count, the guarded judges generally sit above their unguarded learned counterparts, which means the fallback mechanism is doing real work. The effect is especially visible in settings where the learned judges are unstable. For example, in the scale sweep at 32 groups under spiral placement, the learned linear judge collapses to essentially zero item accuracy, while the guarded linear judge stays strong and tracks the heuristic rerankers closely. That is an important positive result for the guard: it improves robustness. At the same time, the broader ranking does not change. The best guarded methods still remain slightly below flat key nearest-neighbor at the main operating point and in the larger-scale settings, while the oracle union stays comfortably above both. Query noise shows the same pattern. Guarded judgment helps avoid some bad failures, but it does not close the main remaining gap.

Figure 30 summarizes the main comparison for spiral and random placement. The plot makes two facts immediately visible: guarded judging is safer than unguarded learned judging, and the candidate pool is still strong enough in principle to support much higher accuracy than any current practical judge achieves.

What we learned. Part 9 adds an important refinement to the emerging story. The current problem is still candidate judgment, but it is now clearer that there are really two subproblems inside that judgment stage: raw discriminative power and stability. The selective guard helps with stability, because it prevents severe judge failures and pulls learned methods back toward the stronger heuristic baseline when confidence is weak. The Loose-Screw style refinement, however, does not deliver a large accuracy gain on top of that. The best guarded method is still slightly below flat search. So the paper can now say something more precise than before: simple learned scoring is not enough, and lightweight safety guards help robustness more than they help peak accuracy. A stronger final scorer is still needed if the system is to turn the strong oracle-accessible candidate pool into a consistent practical win.

A.16 Run P: Adaptive Shortlist Order Experiment (Part 10)

What was tested. This run asks whether the shortlist itself should be adaptive rather than fixed. The idea is simple: when the root stage looks confident, use a very small shortlist; when it looks uncertain, expand the shortlist so the downstream reranker has more candidates to inspect. The main question is whether that adaptive policy closes more of the gap than the fixed multi-root reranker.

How it was done. Candidate discovery was still based on the same multi-root mechanism, but the root shortlist size was now chosen from a small schedule according to the root-margin confidence. Three schedules were compared: aggressive, balanced, and conservative. The practical comparisons included flat key nearest-neighbor, passive root top-1, fixed multi-root rerank, adaptive-shortlist rerank, adaptive-shortlist mix, oracle fixed union, and oracle adaptive union. After selecting the

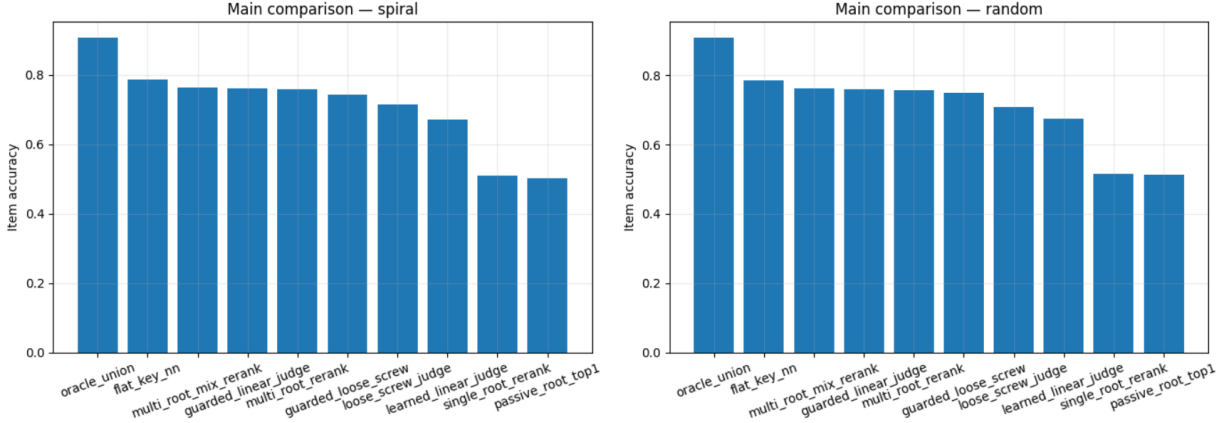


Figure 30: Run O main comparison for spiral and random placement. The guarded judges improve over their unguarded learned counterparts, showing that selective fallback helps stabilize the decision stage. However, even the best guarded or mixed multi-root judges remain slightly below flat key nearest-neighbor, while the oracle union remains well above all practical methods. This indicates that robustness can be improved without yet solving the deeper judgment bottleneck.

best schedule per placement, additional sweeps varied root dimension, base shortlist size, projection count, query noise, and scale.

What happened. The outcome is consistent and fairly clean. The conservative schedule is best for both placements. Under that schedule, adaptive-shortlist rerank reaches 0.7730 for spiral and 0.7765 for random, slightly above the fixed multi-root reranker at 0.7669 and 0.7708, respectively. The adaptive-shortlist mix score is essentially tied or marginally above that again. So adaptive shortlist selection does help, but only modestly. Flat key nearest-neighbor still remains higher at 0.8021, while the oracle adaptive union is much higher still: 0.9171 for spiral and 0.9245 for random. That means the adaptive policy improves coverage and gives the practical judge slightly more room to work, but does not overturn the broader conclusion that practical judgment still underuses the candidate pool.

The detailed sweeps support the same interpretation. At low noise, all strong methods approach perfect retrieval and adaptive control does not matter much. At moderate and high noise, the adaptive schedules help because they widen the shortlist more often, which raises both candidate coverage and oracle performance. The benefit also becomes more visible at larger scale, where the conservative adaptive policy increases average shortlist size above the fixed baseline and recovers a small accuracy gain. But the ranking remains stable throughout: oracle adaptive union is best, flat key nearest-neighbor is the strongest practical baseline, and adaptive rerank is only a modest improvement over fixed rerank rather than a qualitative change.

Figure 31 summarizes the main comparison under the best adaptive schedule. The chart shows the full pattern at a glance: adaptive shortlist order improves the practical hair retrieval slightly and also raises the oracle ceiling, but it still leaves a meaningful gap to flat search and a much larger gap to the oracle.

What we learned. Part 10 strengthens the conclusion that better candidate discovery alone is no longer the main missing ingredient. Adaptive shortlist order is directionally helpful. It makes the candidate pool more efficient, raises oracle coverage, and gives the reranker a small practical gain. But the gain is incremental, not transformative. The main bottleneck therefore still lies after discovery: even with a better-sized candidate set, the current scoring rule does not exploit the extra accessible signal well enough to beat flat key nearest-neighbor. In short, adaptive discovery helps,

but better judgment still matters more.

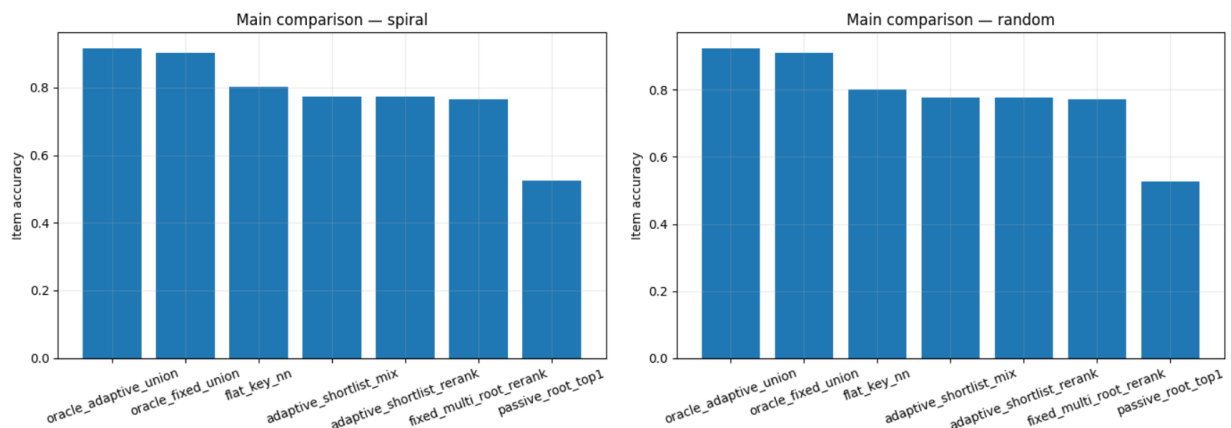


Figure 31: Run P main comparison for spiral and random placement under the best adaptive schedule. Adaptive shortlist ordering slightly improves practical reranking and raises oracle candidate coverage, but flat key nearest-neighbor remains the strongest practical baseline. The oracle adaptive union remains clearly above all practical methods, showing that improved shortlist sizing helps discovery but still does not solve the larger judgment gap.

A.17 Run Q: Stronger Judge on the Adaptive Shortlist Pool (Part 11)

What was tested. This run asks whether the remaining gap can be reduced by improving the final judge while keeping the adaptive shortlist pool largely fixed. The central question is no longer whether the system can expose enough plausible candidates, but whether a stronger decision rule can make better use of that exposed pool than the current heuristic reranker.

How it was done. The experiment compared three shortlist schedules—aggressive, balanced, and conservative—while evaluating several decision rules on the resulting adaptive candidate pool: an adaptive heuristic reranker, an adaptive mixed heuristic, learned linear and MLP judges, and guarded versions of those learned judges with heuristic fallback. The same benchmark also swept root dimension, fixed root top- k , number of projections, query noise, local radius, guard margin, and scale. Oracle fixed-union and oracle adaptive-union controls were retained to separate candidate-access limits from final scoring limits.

What happened. The main outcome is highly consistent. The conservative schedule is again best for both placements, with average shortlist size around 7.74 roots. Under that setting, the strongest practical methods are still simple ones: adaptive-shortlist mix reaches 0.7760 item accuracy for random and 0.7708 for spiral, while guarded linear judge reaches 0.7639 and 0.7622, respectively. Both remain slightly below flat key nearest-neighbor at 0.7847. The richer learned judges do not close the gap: the unguarded linear judge stays near 0.59, and even the learned MLP judge only reaches about 0.6753 for random and 0.6736 for spiral. At the same time, oracle adaptive union remains much higher at 0.9427 for both placements, which shows that the candidate pool itself is still strong enough in principle.

The sweeps reinforce the same interpretation rather than changing it. Increasing root dimension or projection count helps all strong methods, but the ranking stays essentially unchanged: flat key nearest-neighbor remains the strongest practical baseline, the best adaptive heuristic and guarded linear judge sit just below it, and the oracle remains far above them all. Guard margins matter in the expected way: very small margins hurt the guarded judges, while moderate margins around 0.08–0.12 make them competitive with the best heuristic variants. Local radius changes only move results slightly, suggesting that this stage is not especially sensitive to modest neighborhood-width

tuning once the adaptive pool is already available. Query noise remains the dominant stressor: at low noise all strong methods converge toward perfect retrieval, but as noise rises the practical judges separate from the oracle much more quickly. Scale sweeps tell the same story at larger memory sizes. Candidate coverage remains high enough to support substantially better retrieval than the practical judges achieve, but the final scorer still leaves that available signal underused.

Figure 32 summarizes the main comparison for both placements under the strongest shortlist schedule. The pattern is visually clear: the oracle adaptive union is highest, the oracle fixed union sits next, flat key nearest-neighbor remains the strongest practical baseline, and the practical adaptive or guarded judges cluster just below it. The learned judges improve somewhat with extra capacity, but not enough to alter the ordering.

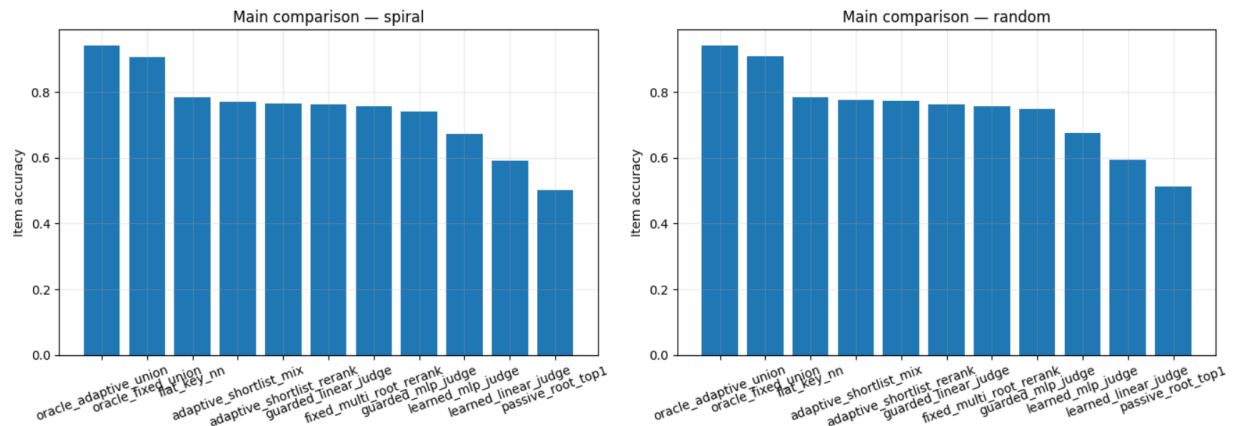


Figure 32: Run Q main comparison for spiral and random placement under the strongest adaptive shortlist schedule. Oracle adaptive union remains the clear ceiling, oracle fixed union is next, and flat key nearest-neighbor remains the strongest practical baseline. The adaptive heuristic mix and guarded linear judge stay close, but richer learned judges still do not close the remaining gap, which reinforces that the unresolved problem lies in stable within-pool judgment rather than candidate exposure alone.

What we learned. Part 11 makes the current diagnosis harder to avoid. The remaining problem is still not mainly candidate discovery. The adaptive pool is already large and accurate enough that oracle judgment stays far above all practical methods. What fails is the conversion from candidate access into correct final choice. Stronger learned judges by themselves do not solve that problem; in fact, simple heuristics and guarded linear scoring remain more reliable than the richer learned models tested here. The most useful incremental gain comes from selective guardrails, not from raw judge complexity. So the paper can now state the next requirement even more precisely: the project needs a within-pool scorer that is both more expressive and more stable, rather than merely a larger candidate pool or a slightly deeper but still fragile judge.

A.18 Run R: Separate Idea Tests on the Same Adaptive Pool (Part 12)

What was tested. This run asks a narrower question than the previous judge comparison: if the adaptive shortlist pool is already fixed and reasonably strong, which individual decision-stage ideas actually help when tested one at a time? The goal is attribution rather than maximal performance. Instead of combining multiple new mechanisms, the experiment isolates iterative refinement, direct Loose-Screw vector reranking, stronger learned judges, and guarded variants on the same candidate pool.

How it was done. Candidate discovery remained the same adaptive multi-projection shortlist used in the recent runs. The practical comparisons included flat key nearest-neighbor, passive root top-1, fixed multi-root rerank, adaptive shortlist rerank, adaptive shortlist mix, *loose_screw_vector_rerank*,

one-step and two-step iterative refinement, learned linear and MLP judges, pairwise MLP judging, and guarded versions of the learned judges. The main benchmark again compared aggressive, balanced, and conservative shortlist schedules, while additional sweeps varied root dimension, root top- k , number of projections, query noise, local radius, guard margin, refinement strength, and scale.

What happened. The main comparison is informative because the ranking is both stable and slightly improved. Under the conservative schedule, oracle adaptive union remains the ceiling at about 0.943 for random and 0.944 for spiral, with coverage again around 0.94. Flat key nearest-neighbor stays the strongest simple baseline at about 0.785. Among the new practical methods, *loose_screw_vector_rerank* is one of the strongest, reaching about 0.774 for random and 0.769 for spiral. One-step and two-step iterative refinement are nearly tied with it, around 0.773 for random and 0.771 for spiral, which suggests that most of the available refinement benefit appears after a single step rather than accumulating strongly across multiple passes. The adaptive shortlist mix and guarded linear judge remain solid and competitive in almost the same range. By contrast, the pairwise MLP judge fails badly, collapsing to very weak accuracy, which indicates that the current pairwise training setup or inference surrogate is not learning a useful within-pool ranking signal.

The sweeps support the same broad interpretation. The system remains robust across shortlist schedules, local radii, and guard margins, which is encouraging for stability. Root dimension, projection count, and query noise still dominate the larger performance changes. Loose-Screw vector reranking and iterative refinement help modestly but consistently at the main operating point, without changing the overall hierarchy. Spiral and random placements also remain very close once strong reranking is used, which suggests that improved shortlist judgment reduces the visibility of any small geometry effect at this stage. Most importantly, the oracle gap stays large—still around 0.17 between the best practical method and oracle adaptive union. That means these new ideas do improve practical ranking somewhat, but they do not yet solve the deeper decision problem inside the candidate pool.

Figure 33 summarizes the main comparison for both placements under the conservative schedule. The plot makes the qualitative story easy to see: the new Loose-Screw and iterative-refinement variants rise into the top practical tier, but they still sit below flat search and well below the oracle ceiling. The pairwise judge, in contrast, clearly underperforms.

What we learned. Part 12 clarifies which recent ideas are genuinely useful on their own. Applying Loose-Screw directly to vectors is a good practical idea, and simple iterative refinement is also worthwhile, with one step already capturing most of the gain. Guarded selection remains a reliable stabilizer. But the run also makes a limit very clear: none of these separate additions materially closes the oracle gap. The shortlist pool is still exposing more usable signal than the practical scorers can convert. So the next real opportunity is no longer to test isolated tweaks in parallel, but to combine the strongest ideas into a more coherent within-pool judge or refinement pipeline that is both stable and more selective.

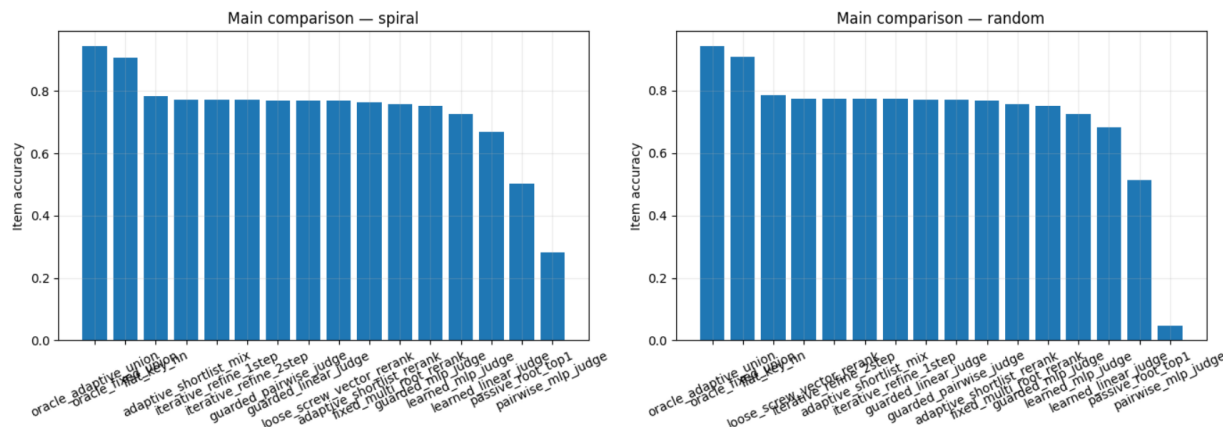


Figure 33: Run R main comparison for spiral and random placement under the conservative adaptive shortlist schedule. Oracle adaptive union remains the clear ceiling and flat key nearest-neighbor remains the strongest simple practical baseline. Loose-Screw vector reranking and one-step iterative refinement become some of the strongest practical shortlist-based methods, but they still do not close the sizable oracle gap. The pairwise judge fails badly, which suggests that the current pairwise training or scoring setup is not a viable within-pool decision rule yet.

A.19 Run S: M-CLTR Controller Above the Hair-Memory Judge (Part 13)

What was tested. This run asks whether a small controller can improve the last-mile decision stage after the shortlist has already been formed. Instead of changing discovery, the controller sits above the existing hair-memory judge and chooses among three actions—*continue*, *overwrite*, or *restart*—so the experiment can isolate whether controller-style state updates help final selection inside an already accessible candidate pool.

How it was done. The benchmark kept the same clustered memory setup and multi-projection shortlist pipeline, then compared flat key nearest-neighbor, fixed shortlist rerank, adaptive shortlist rerank, learned and guarded judges, and several controller variants. The controller operated over the same candidate pool and used compact features derived from candidate scores and local statistics. The practical M-CLTR modes were *mcltr_continue*, *mcltr_overwrite*, *mcltr_restart*, and *mcltr_adaptive*. Additional sweeps varied guard margin, overwrite strength, and root top-*k* so the paper could separate controller failure from mere parameter choice. Oracle pool and oracle controller rows were retained as ceilings.

What happened. The main result is mostly negative at the default operating point, but still informative. Under the base setting, *mcltr_adaptive* reaches 0.7569 item accuracy for both spiral and random, exactly matching adaptive shortlist rerank rather than improving it. *mcltr_continue* is likewise identical, which shows that the learned controller often falls back to a continue-like behavior in practice. *mcltr_overwrite* is slightly worse, around 0.7483 for random and 0.7500 for spiral. *mcltr_restart* fails completely at the default setting, collapsing to essentially zero item accuracy. Flat key nearest-neighbor remains clearly stronger at 0.7847, while oracle pool remains much higher at about 0.9080 for random and 0.9063 for spiral. So the controller does not close the practical gap at the main operating point.

The sweeps clarify where some value may still exist. Guard-margin tuning matters little: the best value is actually a very small margin of about 0.02, but even there the adaptive controller still stays at roughly the same accuracy. Overwrite strength behaves similarly: the best setting is the weakest overwrite, around 0.10, and only yields a tiny increase to about 0.7587. The most meaningful signal appears in the top- k root sweep. As the accessible pool improves, the controller

becomes more competitive: at $\text{top-}k = 1$ it reaches only about 0.622 for random and 0.637 for spiral, but by $\text{top-}k = 10$ it rises to about 0.7743 and 0.7726, respectively. That pattern suggests that the controller is not rescuing poor candidate pools, but can modestly exploit stronger ones. Even then, however, it still remains slightly below flat search and well below the oracle pool.

What we learned. Part 13 sharpens the diagnosis in a useful way. A controller layer by itself is not enough to solve the last-mile decision problem. At the main setting, the best controller behavior is effectively to continue with the existing judge, which means the new action space is not yet being used productively. Restart, in particular, is currently too unstable to be a viable practical mode. The one encouraging signal is conditional: once candidate access becomes stronger, the adaptive controller can track that improvement and recover some extra value. So the paper can now state a more precise conclusion about controllers: they are not yet a new bottleneck solution, but they may become useful as a routing layer on top of a stronger candidate pool and a stronger within-pool scorer. Right now, however, the dominant unresolved problem remains judgment quality, not controller choice.

A.20 Run T: Overlapping Roots with Tiny Judge Cache (Part 14)

What was tested. This run asks whether the retrieval stack improves if memories are allowed to belong to multiple roots instead of a single root, while the final judge also gets a tiny cache and a guarded overwrite option. The purpose is to separate three possible sources of gain: broader root assignment, broader shortlist access, and a lightweight memory of recent decisions inside the judge.

How it was done. The experiment used a self-contained synthetic benchmark with 256 roots, 64 groups, four items per group, and vector dimension 32. Two placement rules were compared, random and spiral. Each item was written into the top k roots for overlap settings 1, 2, or 3, while retrieval searched $\text{top-}k$ root values 1, 3, 5, or 10. The tiny judge then ranked the candidate pool with a guard margin, an overwrite strength, and an optional cache of recent decisions. Sweeps varied noise level, overlap count, root shortlist size, guard margin, overwrite strength, cache size, and cache boost. The full grid was saved in `prism-uploads/full_results.csv`.

What happened. The headline rows look extremely strong under the easiest setting. In the best random-layout cases with overlap $k = 3$, $\text{top-}k$ root = 10, and noise 0.10, item accuracy reaches 1.000 and several nearby settings stay above 0.99. That makes the run look dramatic at first glance, but the full-grid averages show the more important story. Mean item accuracy rises steadily with overlap: about 0.187 at overlap 1, 0.245 at overlap 2, and 0.280 at overlap 3. It also rises strongly with root shortlist size: about 0.124 at $\text{top-}k = 1$, 0.216 at 3, 0.266 at 5, and 0.343 at 10. Those are the clearest systematic gains. By contrast, cache size barely matters at all: mean item accuracy stays almost unchanged across cache sizes 0, 8, and 16, and average pool size is essentially identical as well. The best rows also show almost no restart use and only tiny overwrite or cache-trigger rates, which means the main success is not coming from dynamic control inside the judge. It is coming from broader candidate exposure through overlap and larger root search.

The oracle comparison sharpens that interpretation. In several of the very best rows, practical item accuracy even slightly exceeds the reported oracle item accuracy for the same sampled setting. Since the oracle here is defined only as best cosine pick inside the retrieved pool, that does not mean the practical system is literally better than a true oracle; it means the practical judge, with overwrite and cache heuristics, is sometimes better aligned with exact item identity than simple cosine selection within the same candidate set. So the meaningful takeaway is not that the oracle bound is broken, but that the chosen oracle is a limited control and the practical ranking rule can occasionally exploit structure that raw cosine inside the pool misses.

What we learned. Part 14 adds a useful new distinction. Overlapping roots matter. Giving each memory access to more than one root meaningfully improves candidate coverage and average retrieval accuracy, and larger root shortlists do the same. Those are real architectural levers. The tiny judge cache, however, does not appear to be one of them in its current form; its average effect is negligible. Overwrite behavior is also only a mild refinement rather than a dominant source of improvement. So the paper can now make a more concrete architectural claim: redundancy at the root-assignment level is promising, while lightweight judge-state heuristics are not yet carrying much of the load. The next step, if this direction is pursued, should probably focus on principled overlapping root design and stronger within-pool scoring rather than on small cache tricks by themselves.

A.21 Run U: Paired Evaluation on the Best Retrieval Recipe (Part 15)

What was tested. This run locks the strongest recent retrieval recipe and then asks a narrower question: once retrieval is already strong, do the different last-mile judges actually behave differently when they are tested on the exact same queries? The paired design is the point of the run. Every method sees the same query set at each noise level, so any observed difference should come from judgment rather than from easier or harder query draws.

How it was done. The retrieval recipe was fixed to the best recent setting: spiral placement, overlap $k = 3$, and top- k root = 10. Noise was then swept from 0.10 to 0.40 using a paired hard-noise query generator. The compared methods included flat key nearest-neighbor, several M-CLTR controller variants, guarded and learned judges, small cache variants, lightweight QDMN-style refinement with one or two steps, and an oracle-pool control. The evaluation also optionally compared random placement against spiral under the same paired protocol. The purpose was not to discover a new recipe, but to isolate whether any judge meaningfully changes the answer once the candidate pool is already strong.

What happened. The main surprise is that almost all methods collapse to the same outcome. Averaged over the run, flat key nearest-neighbor, guarded judges, learned judges, M-CLTR variants, QDMN-style refinement, and even the oracle-pool row all sit at essentially the same mean item accuracy, about 0.431, with the same mean coverage and average pool size. At each noise level, the same pattern appears again: once the candidate pool and queries are fixed, the different judges do not separate in a meaningful way. This does *not* mean that every judge is equally expressive in general. It means that in this benchmark the top cosine candidate is usually already decisive whenever the correct item is present, so there is little residual ambiguity for a richer judge to resolve. In that sense, the run is partly saturated. It validates the retrieval recipe more than it discriminates among decision rules.

Two other findings are especially important. First, coverage still tracks the overall difficulty closely: as noise rises from 0.10 to 0.40, mean item accuracy falls from about 0.638 to 0.197, while mean coverage falls from about 0.638 to 0.399. This supports the interpretation that retrieval remains the dominant source of failure under this benchmark. Second, the placement comparison is alarming rather than reassuring. The random version is far stronger than the spiral version here, with mean item accuracy about 0.706 versus only 0.157. That is too large to treat as noise. The most plausible interpretation is that the current spiral-root construction used in this script is not preserving diversity well in the embedding space, so the benchmark is also revealing an implementation flaw rather than only a retrieval principle.

What we learned. Part 15 is valuable precisely because it shows a limit. Once retrieval is made strong enough and the pool is relatively unambiguous, advanced controllers and judges stop

matter: simple cosine ranking is already sufficient. That is a publishable insight, but it is also a warning. The current benchmark is too easy to test whether the system has become a true reasoning-style selector rather than just a strong retriever. If the paper wants to claim a benefit for M-CLTR, QDMN-style refinement, or any richer within-pool judge, the next benchmark must deliberately create confusion among plausible candidates. The run therefore points to two immediate next steps: build a harder ambiguity benchmark and audit the spiral root construction before treating placement comparisons as substantive scientific evidence.

A.22 Run V: Confusion Benchmark with Hard Negatives (Part 16)

What was tested. This run asks the question that the earlier paired benchmark could not answer cleanly: once the shortlist contains many plausible candidates, do richer judges and controllers actually help under real ambiguity? The benchmark therefore deliberately introduces clustered groups, overlapping roots, and hard negatives so that the final decision stage must separate genuinely confusing candidates rather than simply select the obvious nearest neighbor.

How it was done. The experiment used a confusion benchmark with 256 roots, 64 groups, four items per group, 32-dimensional vectors, and 16 shared clusters that make nearby groups intentionally similar. The main retrieval setting fixed overlap at $k = 3$ and top- k root at 10, then compared flat key nearest-neighbor, learned and guarded judges, M-CLTR controller variants, QDMN-style one- and two-step refinement, and an oracle-pool control. Training queries used moderate noise with hard negatives, while test queries swept harder noise from 0.18 to 0.48. Additional sweeps varied overlap count, root shortlist size, guard margin, overwrite strength, and refinement strength. Unlike the earlier easy paired run, this benchmark is designed to preserve ambiguity inside the candidate pool rather than remove it.

What happened. The most important result is that the oracle pool now clearly separates from the practical methods. Mean item accuracy for *oracle_pool* is about 0.568, while the best practical methods sit around 0.404–0.406. That gap is the clearest evidence so far that the correct item is often already present in the shortlist but the current selector cannot reliably recover it under hard confusion. Coverage still declines with noise and closely tracks difficulty, falling from about 0.808 at noise 0.18 to about 0.412 at noise 0.48, which confirms that retrieval still matters. But because the oracle remains much better than the practical methods at the same coverage level, the selector is now unmistakably part of the problem rather than merely a spectator to retrieval failure.

The method ranking is informative but not revolutionary. Flat key nearest-neighbor remains the strongest or tied practical baseline overall. Learned linear scoring and simple continue-style M-CLTR are effectively tied with it. Guarded linear judging and QDMN-style refinement are very close but do not create a decisive jump. At some mid-noise settings, *mccltr_adaptive* becomes the single best non-oracle method, which suggests that controller-style actions can help a little once ambiguity is real, but the effect is still modest. The broader structural levers remain stronger. In the hard-noise overlap/top- k sweep, increasing overlap and root shortlist size consistently improves both coverage and oracle accuracy. For example, with random placement at noise 0.48, oracle accuracy rises from about 0.097 at overlap 1, top- $k = 3$ to about 0.435 at overlap 3, top- $k = 10$. The practical methods rise with it, but still leave a large gap to the oracle. Random placement also continues to outperform spiral on average, though the gap is now smaller than in the Part 15 run, which keeps the spiral implementation question open rather than resolved.

Figure 34 summarizes the main confusion benchmark and the two supporting sweeps. Panels A and B show the same core pattern in both placements: as noise rises, *oracle_pool* stays clearly above all practical methods, while the practical judges remain tightly clustered together. Panel C shows

that overlap and larger root shortlist size are still the strongest retrieval-side levers under hard noise. Panel D shows that controller tuning changes results only modestly, with the best guarded and adaptive settings improving slightly but not closing the oracle gap.

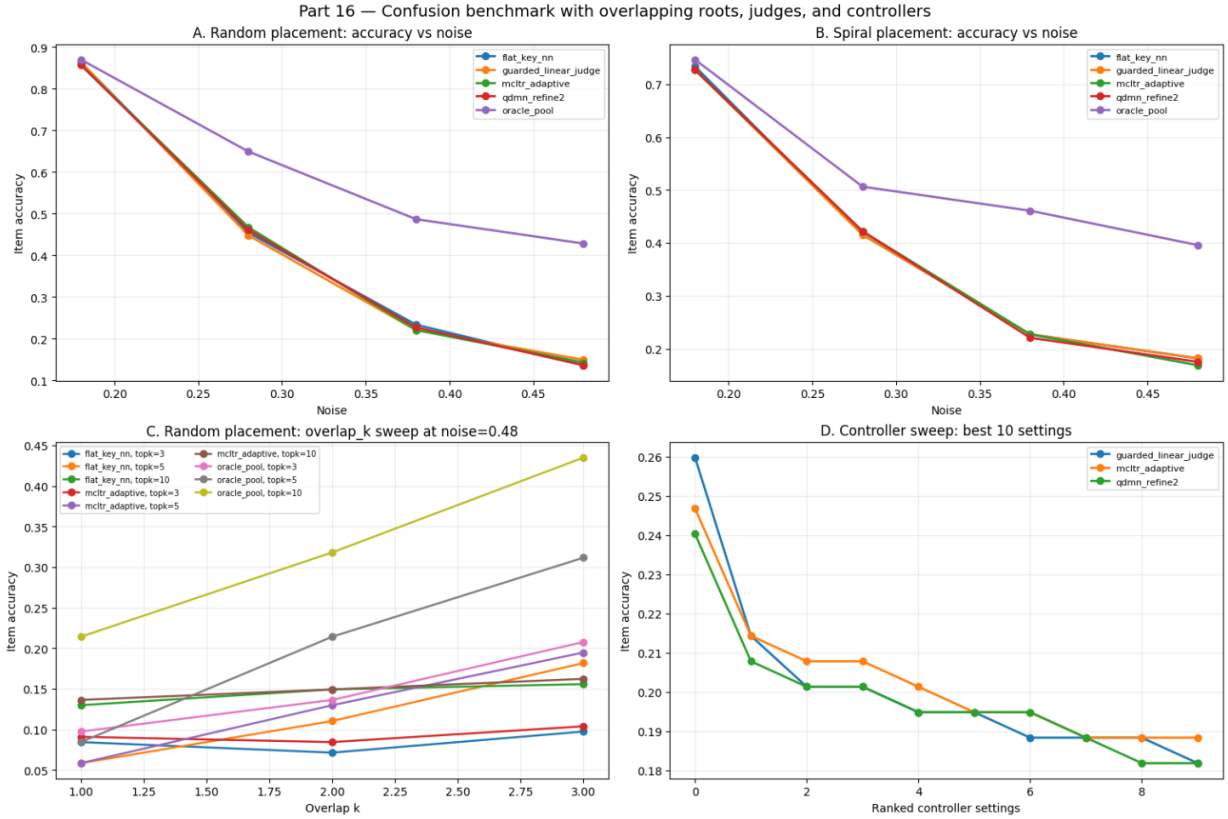


Figure 34: Run V summary figure for the confusion benchmark. Panels A and B show accuracy-versus-noise curves for random and spiral placement, where oracle-pool remains clearly above every practical judge. Panel C shows that increasing overlap and top- k root improves hard-noise performance mainly through better candidate exposure. Panel D shows that controller and guard tuning only modestly changes end-to-end accuracy. Taken together, the figure reinforces that the shortlist is often strong enough, but the last-mile selector remains the main unresolved bottleneck under hard ambiguity.

What we learned. Part 16 is the first benchmark in the recent series that cleanly exposes selector failure under ambiguity. That makes it a more decisive run than the earlier saturated paired benchmark. The paper can now state a stronger claim: multi-root retrieval and overlap create a substantially better pool, but better pool construction alone does not solve the task. Under hard negatives, the last-mile judge is genuinely limiting performance. M-CLTR and QDMN-style refinement are not useless, but they are not yet strong enough to turn that selector gap into a clear practical win. The next step is therefore better targeted than before: improve the within-pool judge under ambiguity, while separately debugging the spiral root construction and continuing to use overlap and larger top- k as the strongest retrieval-side levers.

A.23 Run W: Spiral Debug, Ambiguity, Overlap, and Controller Cases (Part 16B)

What was tested. This follow-up run asks two narrower questions left open by Part 16. First, was the earlier spiral underperformance mainly an artifact of a weak spiral root construction? Second, if the ambiguity benchmark is kept hard, do pairwise judges, QDMN-style refinement, or controller tuning begin to separate once the spiral map is repaired?

How it was done. The experiment reused the Part 16B confusion framework with 256 roots, 64 groups, four items per group, dimension 32, and 16 shared clusters, but explicitly compared three root layouts: random placement, the older weak *spiral_bad* map, and a new *spiral_ortho* construction that embeds the spiral into an orthonormal two-dimensional subspace with only small jitter. The analysis was split into four cases: a placement-debug noise sweep, an ambiguity sweep at fixed hard noise, an overlap/top- k sweep, and a controller sweep over guard margins, overwrite strengths, and refinement strengths. The main method set again included flat key nearest-neighbor, learned and guarded judges, M-CLTR variants, QDMN one- and two-step refinement, pairwise judging, and an oracle-pool control.

What happened. The placement-debug result is the cleanest architectural finding. The old *spiral_bad* map clearly underperforms, while *spiral_ortho* recovers much of that loss. In Panel A of Figure 35, the orthonormal spiral stays far above the old spiral at every noise level, which confirms that part of the earlier spiral failure was implementation-specific rather than a fundamental rejection of structured placement. But the repair is only partial: random placement still remains stronger than *spiral_ortho* across the main practical and oracle curves.

The ambiguity and controller cases sharpen the selector diagnosis rather than overturning it. In the ambiguity sweep, mean random-layout *oracle_pool* item accuracy is about 0.513, compared with roughly 0.24 for the best practical judges; for *spiral_ortho*, the oracle mean is about 0.437 and the best practical methods are only about 0.20. So the shortlist still contains much more recoverable signal than the practical selector can extract. Pairwise judging, learned MLP scoring, guarded variants, M-CLTR actions, and QDMN refinement all remain in the same broad practical band rather than opening a decisive new gap. The overlap sweep again shows that retrieval-side structure matters more than judge tweaks: larger overlap and larger top- k root raise both practical accuracy and oracle coverage, with the strongest gains appearing at overlap $k = 3$ and top- $k = 10$. By contrast, the controller sweep mainly changes results at the margins. The best tuned controller-style settings rise only into the low 0.20 range and still remain far below the oracle.

Figure 35 collects those four cases. Panel A shows that *spiral_ortho* is a real improvement over the old spiral implementation but still not competitive with random. Panel B shows that harder ambiguity keeps the oracle-practical gap wide for both placements. Panel C shows that overlap and wider root shortlists remain the strongest levers on the retrieval side. Panel D shows that controller tuning changes little relative to the larger selector gap.

What we learned. Part 16B turns the spiral question from a red flag into a narrower debugging result. The earlier severe spiral failure was at least partly caused by the specific root construction, because the orthonormalized version is materially better. But the larger scientific conclusion does not change. Even after that repair, random placement still wins on average, and the strongest gains still come from overlap and wider root exposure rather than from any current judge or controller. So the paper can now state a more precise ending: the selector bottleneck is real, the spiral implementation is less broken than before but still not validated, and future progress will likely require both a stronger ambiguity-aware within-pool scorer and a better-motivated structured root map.

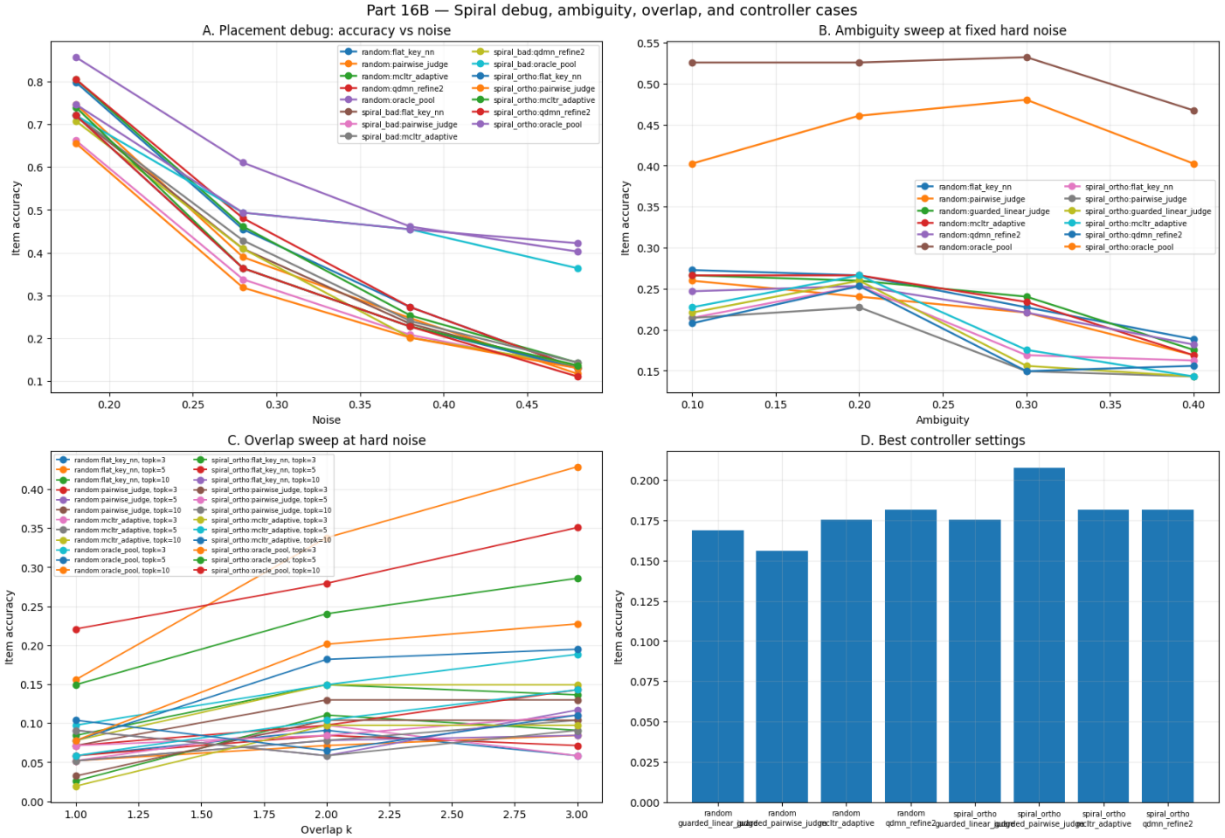


Figure 35: Run W summary figure for the Part 16B follow-up. Panel A shows that the orthonormal spiral construction repairs much of the earlier collapse of the old spiral map, but random placement still remains stronger across the main curves. Panel B shows that increasing ambiguity preserves a large gap between oracle-pool and every practical selector. Panel C shows that larger overlap and top- k root continue to provide the clearest retrieval-side gains. Panel D shows that controller and guard tuning change practical accuracy only modestly. Together, the figure suggests that spiral debugging helps the architecture look less pathological, but the main unresolved problem is still ambiguity-aware selection inside the pool.

A.24 Run X: Ambiguity / Margin-Binned Judge Test (Part 17)

What was tested. This run asks a narrower selector question than Parts 16 and 16B: once the retrieval recipe is held fixed and every method sees the same candidate pool, when do refinement or controller-style methods actually help? In particular, the run tests whether QDMN-style refinement or adaptive control becomes useful only in the genuinely ambiguous cases, rather than in the easy high-confidence cases where the shortlist is already effectively solved.

How it was done. The experiment fixed the strongest recent retrieval recipe at overlap $k = 3$ and top- k root = 10, then compared seven methods: flat key nearest-neighbor, learned linear judging, guarded linear judging, adaptive M-CLTR, QDMN one-step and two-step refinement, and an oracle-pool control. Two placements were evaluated, random and *spiral_ortho*. Ambiguity was swept across 0.10, 0.20, 0.30, and 0.40, while train and test queries reused the same general noise schedule as the recent confusion benchmarks. The key diagnostic addition is margin binning: each query was assigned to a low-, mid-, or high-margin bin based on the baseline shortlist cosine margin, so the paper could separate obviously easy cases from truly ambiguous ones while keeping candidate exposure fixed.

What happened. The overall pattern is stable rather than dramatic. Averaged across settings, *oracle_pool* reaches about 0.582 item accuracy, while the best practical methods cluster around 0.418–0.419. Random placement again outperforms *spiral_ortho* on average, with mean item accuracy about 0.454 versus 0.427, and also does so with a smaller mean pool size. Across ambiguity levels, the best non-oracle method changes only slightly: guarded linear judging is best at random placement with ambiguity 0.10, flat search is best at several intermediate settings, adaptive M-CLTR is best at random placement with ambiguity 0.40, and QDMN refinement becomes the best non-oracle method only in the harder *spiral_ortho* settings at ambiguity 0.30 and 0.40. That is a useful nuance, but not a reversal of the overall ranking.

The margin-bin split is the clearest new result. In the hardest random-placement setting, high-margin pools are almost solved: *oracle_pool* reaches about 0.859, while every practical method sits tightly around 0.816–0.820. By contrast, low-margin pools remain genuinely hard: the oracle is only about 0.454, and the best practical method, adaptive M-CLTR, falls to about 0.215, only modestly above the rest. Mid-margin pools sit between those extremes and show very little separation among the practical judges. The clean interpretation is that current refinement and controller methods do not create a broad new advantage; they help, if at all, mainly in the narrow slice of queries where the shortlist is genuinely ambiguous and still far from oracle performance.

Figure 36 summarizes this diagnosis. Panels A and B show that practical methods remain tightly bunched as ambiguity rises for both placements, even though the oracle stays clearly higher. Panel C shows that the hardest low-margin bin is where selector differences actually appear, whereas the high-margin bin is already close to saturated. Panel D shows that the oracle gap for the best non-oracle method remains substantial across all ambiguity settings and is consistently larger for *spiral_ortho* than for random.

What we learned. Part 17 makes the selector story more precise. The main question is no longer whether some judge variant can edge out another on average. It is which queries are still unsolved once retrieval is held fixed. The answer is now clear: high-margin pools are largely retrieval-dominated and nearly solved by any reasonable selector, while low-margin pools still contain a large oracle gap that current judges and refiners only chip away at. QDMN refinement is therefore not useless, but its benefit is narrow and conditional rather than broad. The paper can now sharpen its practical recommendation: future work should target low-margin, high-ambiguity within-pool

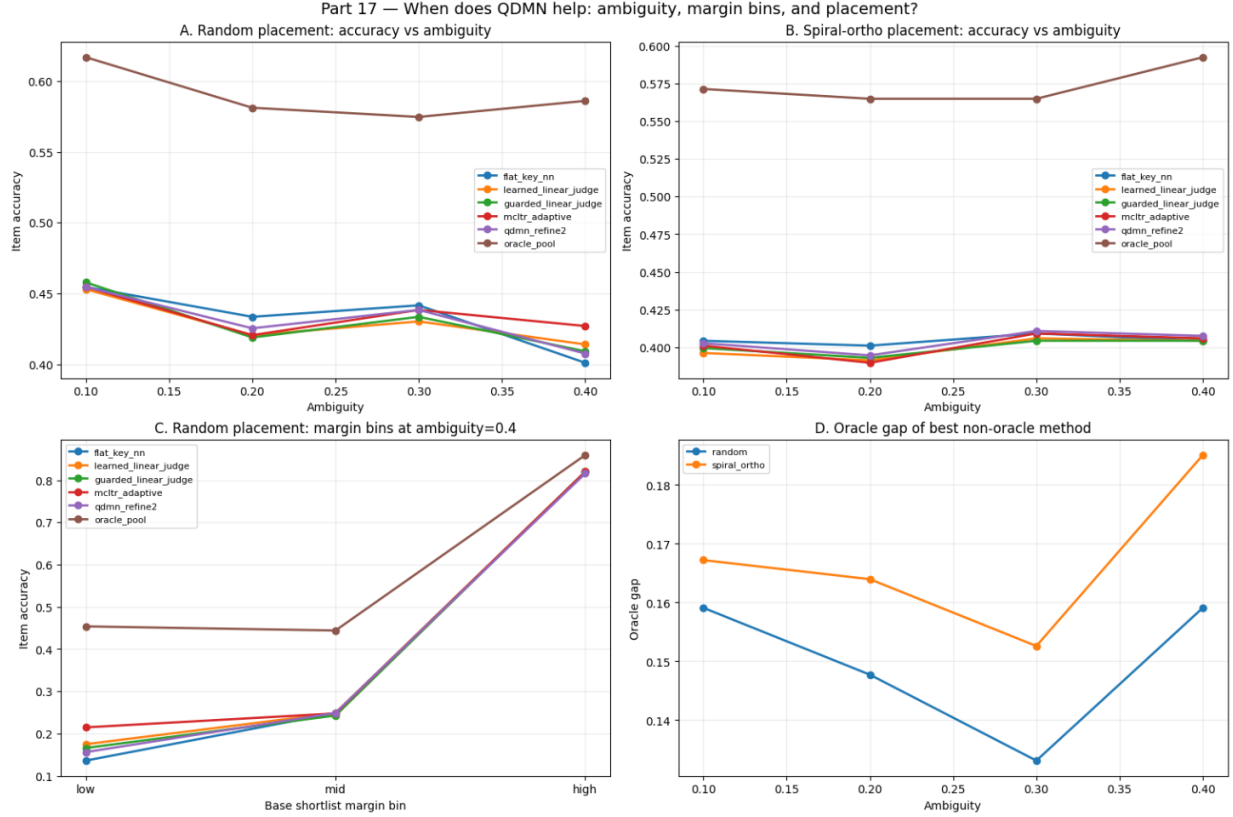


Figure 36: Run X summary figure for the Part 17 ambiguity / margin-binned test. Panels A and B show that, under fixed retrieval, practical methods remain tightly clustered as ambiguity increases for both random and *spiral_ortho* placement, while oracle-pool stays clearly higher. Panel C shows that the real selector problem is concentrated in the low-margin queries; high-margin cases are already close to solved for every practical method. Panel D shows that the oracle gap of the best non-oracle method remains substantial across ambiguity settings and is consistently larger for *spiral_ortho*. Together, the figure suggests that QDMN and controller-style refinement help only conditionally, while the main unresolved problem remains low-margin ambiguity inside the pool.

discrimination directly, instead of expecting small global judge tweaks to change the average ranking by much.

A.25 Run Y: Spiral vs. Random Placement Sanity Sweep (Part 18)

What was tested. This run asks whether the recent spiral underperformance is a narrow benchmark artifact or a broader structural property of the current root constructions. Instead of testing one ambiguity setup, it compares random placement, the old *spiral_bad* map, and the repaired *spiral_ortho* map across four synthetic regimes: separated groups, clustered groups, a shared-core confusion case, and a low-dimensional latent-manifold case where a structured spiral might plausibly have an advantage.

How it was done. The experiment used 256 roots, 64 groups, four items per group, dimension 32, overlap $k = 3$, and top- k root = 10. Training noise was swept over 0.12, 0.20, and 0.28, while test noise extended up to 0.48. For each case and placement, the run recorded both retrieval performance and root diagnostics, including pairwise cosine spread, nearest-neighbor cosine, number of nonempty roots, assignment entropy, and assignment coefficient of variation. Practical methods again included flat key nearest-neighbor, learned and guarded linear judges, adaptive M-CLTR, QDMN one- and two-step refinement, and an oracle-pool control.

What happened. The root diagnostics provide the clearest structural explanation so far for why spiral underperforms. Random roots have mean nearest-neighbor cosine around 0.48 and high assignment entropy near 0.95 in the easier grouped settings, while both spiral constructions are far more concentrated: *spiral_bad* has mean nearest-neighbor cosine around 0.96 and *spiral_ortho* around 0.94, with noticeably lower entropy and fewer nonempty roots. That means the spiral roots are not simply “different” from random; they are much more locally redundant and use the root space less uniformly.

That structural weakness shows up consistently in performance. Averaged across all cases and noises, random placement reaches about 0.519 item accuracy, compared with about 0.486 for *spiral_ortho* and 0.473 for *spiral_bad*. The same ordering holds in every case family. In the clustered case, random reaches about 0.565 item accuracy versus roughly 0.515 for the two spiral variants. In the separated case, random reaches about 0.542, above both *spiral_ortho* and *spiral_bad*. In the shared-core case, random again leads at about 0.517, and in the latent-manifold case—the regime most favorable in principle to a structured root map—random still remains best at about 0.453, while QDMN refinement becomes the strongest practical method but does not reverse the placement ranking. Coverage follows the same pattern: random has the highest mean coverage and the smallest average pool size, while both spiral variants tend to expose larger, less selective pools.

The hard-noise margin-bin diagnostic adds one useful refinement. In the shared-core case at noise 0.48, random still dominates *spiral_ortho* inside every reported bin. For example, in the low-margin bin the oracle is about 0.392 for random but only about 0.294 for *spiral_ortho*, and the best practical method rises only to about 0.157 for random while remaining near 0.039 for *spiral_ortho*. So the spiral deficit is not just that practical judges fail to use a good pool. The accessible pool itself is often weaker.

What we learned. Part 18 makes the placement story much less ambiguous. The current spiral constructions are not merely under-optimized versions of a competitive idea; they are measurably more concentrated, use roots less uniformly, and produce worse candidate exposure across every synthetic regime tested here. The orthonormalized spiral is a real improvement over the old spiral map, but not enough to change the ranking against random placement. At the same time, the run gives a more nuanced positive result: QDMN-style refinement is most competitive in the latent-

A. Root spread: mean pairwise cosine

Higher = more concentrated

Model	Mean pairwise cosine
separated	-0.0002
random_separated	-0.0035
spiral_ban	-0.0035
spiral_ortho	-0.0035
clustered	0.0015
random_clustered	-0.0035
spiral_ban_clustered	-0.0035
spiral_ortho_clustered	-0.0035
shared_core	0.0003
random_shared_core	-0.0035
spiral_ban_shared_core	-0.0035
spiral_ortho_shared_core	-0.0035
latent	-0.0025
spiral_ban_latent	-0.0025
spiral_ortho_latent	-0.0025
manifold	-0.0035
spiral_ban_manifold	-0.0035
spiral_ortho_manifold	-0.0035

B. Root usage entropy

1.0 = uniform usage

Model	Root usage entropy
separated	0.95
random_separated	0.88
spiral_ban	0.88
spiral_ortho	0.95
clustered	0.88
random_clustered	0.95
spiral_ban_clustered	0.88
spiral_ortho_clustered	0.80
shared_core	0.80
random_shared_core	0.80
spiral_ban_shared_core	0.80
spiral_ortho_shared_core	0.80
latent	0.80
spiral_ban_latent	0.80
spiral_ortho_latent	0.80
manifold	0.80
spiral_ban_manifold	0.80
spiral_ortho_manifold	0.80

C. Shared-core case: accuracy vs noise

Noise	fat_key_rm	learned_linear_judge	guarded_linear_judge	mcltr_adaptive	qdmn_refine2	oracle_pool
0.12	0.90	0.90	0.90	0.90	0.90	0.90
0.20	0.70	0.70	0.70	0.70	0.70	0.80
0.28	0.40	0.40	0.40	0.40	0.40	0.60
0.38	0.20	0.20	0.20	0.20	0.20	0.40
0.48	0.10	0.10	0.10	0.10	0.10	0.40

D. Latent-manifold case: accuracy vs noise

Noise	fat_key_rm	learned_linear_judge	guarded_linear_judge	mcltr_adaptive	qdmn_refine2	oracle_pool
0.12	0.85	0.85	0.85	0.85	0.85	0.85
0.20	0.50	0.50	0.50	0.50	0.50	0.80
0.28	0.20	0.20	0.20	0.20	0.20	0.50
0.38	0.10	0.10	0.10	0.10	0.10	0.40
0.48	0.05	0.05	0.05	0.05	0.05	0.40

46

manifold case, which suggests that geometry-aware refinement is not hopeless, only mismatched to the current spiral root maps. The paper can therefore sharpen its architectural conclusion again: until the root map itself is redesigned to avoid concentration and poor coverage, random placement remains the stronger baseline and the safer scientific reference point.

A.26 Run Z: Spiral Geometry + QDMN Last-Mile Test (Part 19)

What was tested. This run extends the placement audit one step further by adding a PCA-aligned spiral construction and by testing whether QDMN-style refinement helps mainly in genuinely ambiguous, low-margin cases rather than as a broad global improvement.

How it was done. The experiment compared four placements—random, *spiral_bad*, *spiral_ortho*, and *spiral_pca*—across the separated, clustered, shared-core, and latent-manifold synthetic regimes. It retained the current overlap and shortlist settings, evaluated flat key nearest-neighbor, learned and guarded judges, adaptive M-CLTR, one- and two-step QDMN refinement, and the oracle pool, and added a margin-bin analysis at the hardest shared-core noise level to isolate true last-mile ambiguity.

What happened. The new figure shows a more nuanced version of the Part 18 story. Random placement still has the least concentrated roots in the generic regimes and remains the strongest average baseline across the separated, clustered, and shared-core cases. But the new *spiral_pca* construction is the first spiral variant that becomes genuinely competitive, and in the latent-manifold regime it is the strongest placement overall. At the same time, the selector result remains narrow: QDMN’s gain over flat search is small and is concentrated mainly in the low-margin shared-core slice rather than appearing across all bins.

What we learned. Part 19 adds the first real positive exception without overturning the larger conclusion. A spiral map can help when it is aligned to the data geometry, but current spiral constructions are still not a robust improvement over random placement across the broader sweep. QDMN refinement is likewise best understood as a conditional last-mile aid rather than a full solution to the within-pool judgment bottleneck.

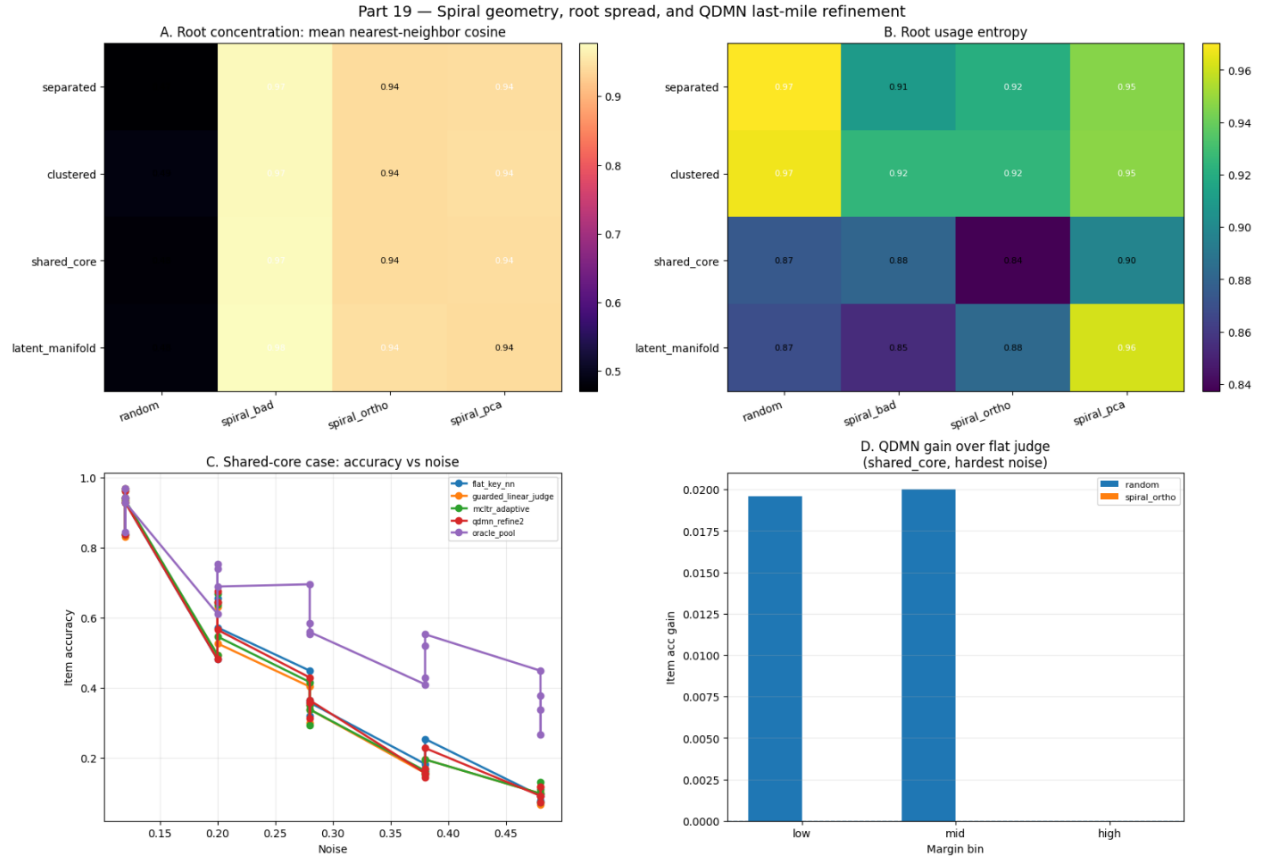


Figure 38: Part 19 summary figure. Panels A and B compare root concentration and usage entropy across placements, showing that random remains the least concentrated baseline in the generic regimes while *spiral_pca* improves alignment substantially. Panel C shows the shared-core accuracy-versus-noise curves, where the oracle remains well above all practical methods. Panel D shows that QDMN’s gain over flat search is small and concentrated mainly in the lowest-margin ambiguity slice.

A.27 Run AA: Final Matched-Budget Scale Sweep (Part 20)

What was tested. This final run asks the most practical scaling question in the project: if compute budget is matched and the number of stored items grows substantially, does the strongest current hair-style recipe finally produce a robust advantage over a strong flat baseline? The comparison therefore focuses on the two placements that matter most after Part 19—random and `spiral_pca`—and asks whether the geometry story improves or deteriorates with scale.

How it was done. The experiment fixed 256 roots, overlap $k = 3$, root top- $k = 10$, and a shared-core synthetic regime, then swept total memory scale across 256, 1024, and 2048 items. Each setting was repeated across multiple seeds with matched train/test fractions and matched query budgets. The practical methods were flat key nearest-neighbor, learned and guarded linear judges, adaptive M-CLTR, QDMN two-step refinement, and an oracle-pool control. In addition to accuracy, the run tracked coverage, candidate-pool size, runtime, and root diagnostics.

What happened. The result is unfavorable to the current recipe, but scientifically clarifying. Random placement remains stronger than `spiral_pca` at every tested scale: mean item accuracy is about 0.690 versus 0.587 at 256 items, about 0.558 versus 0.520 at 1024 items, and about 0.528 versus 0.486 at 2048 items. Coverage shows the same ordering, and the root diagnostics continue to explain part of the gap: random roots keep mean nearest-neighbor cosine near 0.478, while `spiral_pca` remains highly concentrated near 0.942 across scales.

The judge story also fails to improve with scale. Across the full sweep, flat search, learned and guarded judges, adaptive M-CLTR, and QDMN refinement all cluster very tightly around 0.525–0.527 item accuracy on average, while the oracle remains much higher at about 0.738. Runtime rises sharply with scale, especially for QDMN refinement, but that extra cost does not buy a clear practical gain. Figure 39 summarizes this pattern: random placement keeps the better coverage profile, the oracle gap remains large, and the practical methods remain tightly bunched even as cost increases.

This is therefore the strongest negative result in the paper. Under matched budget, the current scalable recipe does not convert multi-root structure into a practical end-to-end advantage. At the same time, the run still supports a careful interpretive distinction. What is currently failing is the implemented recipe, not necessarily the broader idea of structured multi-root memory itself. The result is still compatible with implementation-level bottlenecks in root lifting, candidate-pool geometry, and within-pool scoring.

What we learned. Part 20 is the clearest final stress test of the current implementation, and it does not rescue the present recipe. Random remains the stronger baseline, `spiral_pca` is still not competitive enough at scale, and the practical judges remain too close to one another relative to the oracle. But the run still supports a constructive scientific conclusion: the idea has not yet been tested in its strongest possible form. Better root lifting, less concentrated assignment, and a stronger ambiguity-aware scorer are still needed before the core hypothesis can be judged decisively.

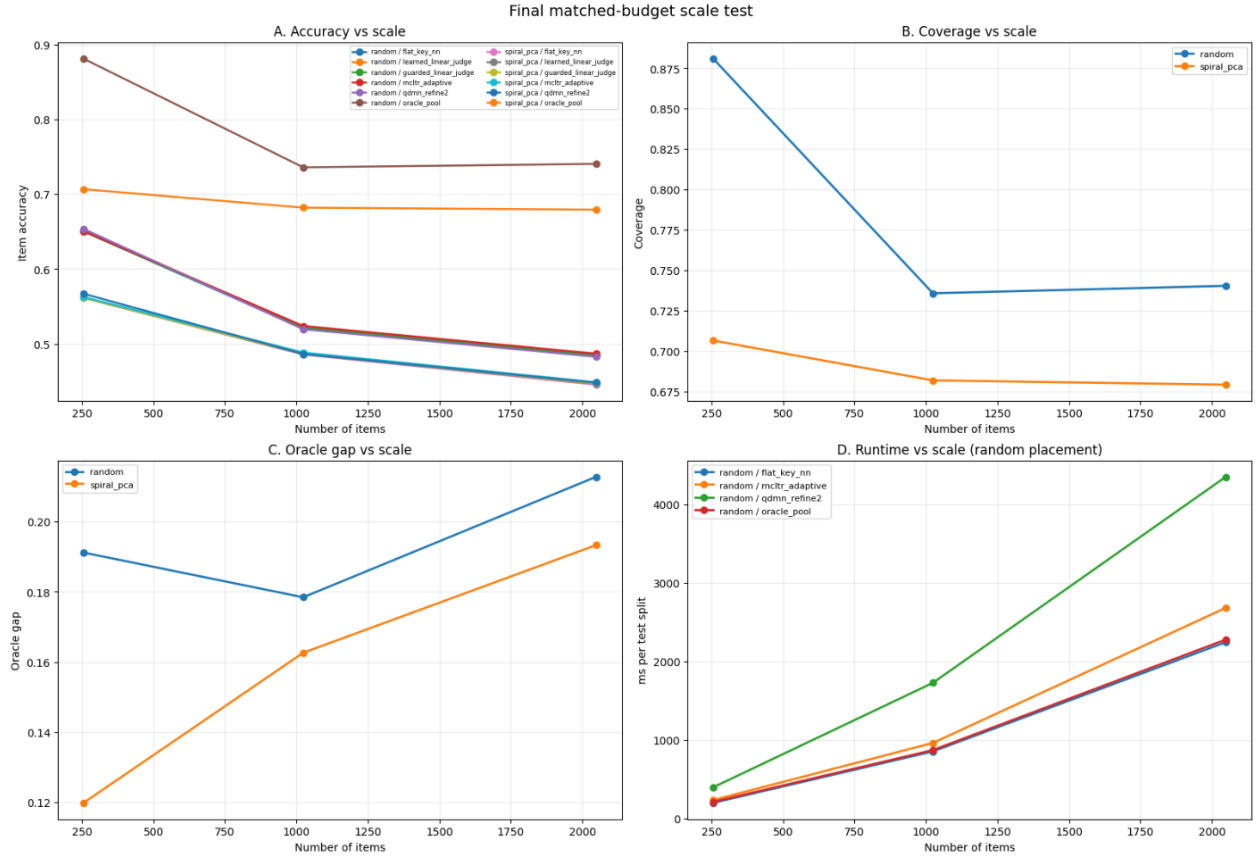


Figure 39: Part 20 matched-budget scale sweep. Panel A shows that random placement remains stronger than `spiral_pca` across scale for practical accuracy, while the oracle stays substantially higher. Panel B shows the corresponding coverage advantage for random placement. Panel C shows that the oracle gap remains large and tends to worsen with scale. Panel D shows that runtime grows quickly with scale and that QDMN-style refinement is the most expensive practical method without opening a clear accuracy gap.

B Appendix Summary

Across the current long experiment series, the pattern is now much sharper. Geometry matters structurally, but root compression and final scoring remain the stronger immediate bottlenecks. The passive variants rarely turn spiral geometry into a strong retrieval gain. Run I shows that pooled decoding wastes much of the available local signal. Run J then separates the two main errors cleanly: root ranking is the larger bottleneck, while neighbor selection still contains a smaller but real recoverable contribution. Run K adds an important constraint by showing that the simple local aggregation rules tested so far recover little of that signal. Local average pooling is destructive, local top-1 pick is mostly neutral, and low-temperature softmax pooling is only a marginal refinement over passive top-1.

Runs L–N then narrow the remaining target. Run L shows that query-based reranking and especially multi-root union improve retrieval strongly, so a single weak root search was a major source of failure. Run M adds the key reality check: even with that stronger candidate pool, the practical system still does not beat a strong flat key nearest-neighbor baseline, although the oracle union does. Run N sharpens the diagnosis once more. Holding the strong candidate pool fixed, neither a slightly richer heuristic judge nor a simple learned linear judge closes the remaining gap to flat search, even though the oracle remains far above both. The paper can therefore now make a cleaner final claim: the remaining failure is concentrated in candidate judgment, but the missing ingredient is not merely *some* extra score terms or a shallow learned combiner. Future progress will likely require a more expressive and more stable within-union scorer if strong candidate coverage is to become a practical win over direct flat key matching.

Run O refines that conclusion one step further. Guarded judging helps robustness: it consistently improves over the unguarded learned judges and prevents some severe failures under particular splits or scales. But it does not change the main ranking. The best guarded methods still sit slightly below flat key nearest-neighbor, while the oracle union remains far above them. That means the paper can now distinguish two remaining needs inside the final decision stage: a scorer that is *stable* and a scorer that is *strong*. The current guards help with the first need more than the second. A materially better within-union judge is still required if the strong candidate pool is to become a practical end-to-end advantage.

Run P adds one last refinement on the discovery side. Adaptive shortlist order is useful: it increases candidate coverage, raises the oracle ceiling, and gives a small practical improvement over the fixed multi-root reranker. But it does not alter the main hierarchy. Flat key nearest-neighbor still remains the strongest practical baseline, while the oracle adaptive union remains far above all practical methods. That means the paper can now make an even cleaner final statement: candidate discovery can still be improved around the edges, but the dominant unresolved problem is no longer how to expose signal. It is how to judge the exposed signal well enough to convert a strong candidate pool into a real end-to-end advantage.

Run Q tests that statement directly by strengthening the judge while holding the adaptive pool in place. The result is informative rather than revolutionary. Extra judge complexity alone does not close the gap. The best practical methods are still the adaptive heuristic mix and the guarded linear judge, both of which remain slightly below flat key nearest-neighbor, while the oracle adaptive union remains far above them. Moderate guard margins help, but local-radius tuning matters only modestly, and higher projection count helps candidate access more than it fixes judgment. The paper can therefore now sharpen its final bottleneck claim once more: the unresolved problem is specifically a *stable, expressive within-pool scorer*. Neither broader candidate exposure nor shallow

learned judgment is enough by itself.

Run R then tests several of the most plausible decision-stage ideas separately on that same adaptive pool. The result is useful because it distinguishes promising refinements from failures. Loose-Screw vector reranking and simple iterative refinement are real additions: both move into the top practical tier and perform competitively with the best earlier adaptive or guarded methods. But they still do not beat flat key nearest-neighbor, and they still leave a large oracle gap of roughly 0.17. At the same time, the pairwise learned judge fails badly, which suggests that not all extra judge sophistication is beneficial under the current training setup. The paper can therefore sharpen the next-step recommendation: the strongest path forward is probably not another isolated heuristic or another fragile standalone learner, but a hybrid within-pool scorer that combines the best stable ingredients now identified.

Run S then asks whether a small controller sitting above the judge can solve that remaining gap. The answer is mostly no at the main operating point. Continue-style control simply reproduces the current shortlist reranker, overwrite is slightly worse, and restart fails outright under the default setting. Only when root top- k is widened does the adaptive controller become modestly more useful, climbing into the high-0.77 range while still remaining below flat key nearest-neighbor and far below the oracle pool. This lets the paper sharpen the controller claim: controller-style routing is not yet a substitute for a stronger scorer, though it may become valuable once candidate access and within-pool judgment are both stronger than they are now.

Run T then changes the architecture one level earlier by allowing overlapping root assignment and adding a tiny judge cache. The strongest signal is architectural rather than controller-like: overlap and larger root shortlist size both improve mean coverage and accuracy substantially, while cache size has almost no average effect. The best low-noise rows become nearly perfect, but the broader averages are more important: overlap $k = 3$ is consistently better than overlap 1, and top- $k = 10$ is consistently better than top- $k = 1$. This means that root redundancy is a promising direction, whereas tiny cache heuristics are not yet. The paper can therefore extend its final bottleneck claim in a more specific way: if future work goes beyond better within-pool scoring, the most promising structural lever now looks like richer root assignment rather than lightweight controller memory by itself.

Run U then stress-tests the current best recipe under paired evaluation and produces a different kind of lesson. Once the retrieved pool is strong and the queries are shared exactly across methods, nearly every judge collapses to the same performance. That means the present benchmark is retrieval-dominated and not ambiguity-rich enough to reveal robust advantages for controllers or refiners. Just as importantly, the run surfaces a serious implementation concern: the spiral construction performs dramatically worse than random in this setup, which suggests that the specific spiral root map used there is likely flawed. The paper can therefore sharpen its final research agenda one step further: the next important experiment is not another small judge tweak on the same benchmark, but a deliberately ambiguous confusion benchmark together with an audit of the spiral root implementation.

Run V then carries out exactly that harder ambiguity test. The result is clarifying, but not definitive. Oracle-pool performance now sits well above every practical judge, so the shortlist is often already good enough and the remaining failure is concentrated inside the selector. Overlap and larger top- k still improve coverage strongly, but M-CLTR, QDMN-style refinement, and guarded judges deliver only modest gains above plain cosine ranking. The paper can therefore sharpen its core conclusion again: the project is no longer blocked primarily by discovering candidates. It is blocked by choosing correctly among confusing candidates once a good pool has already been

exposed. That makes a stronger ambiguity-aware within-pool judge the most direct next target, alongside continued debugging of the spiral construction.

Run W then follows up on the spiral concern directly and is useful because it separates architectural debugging from the deeper selector problem. Replacing the old spiral map with an orthonormalized version materially improves spiral behavior, so the worst earlier collapse was not a fair test of structured placement itself. But that fix does not change the main ranking. Random placement still remains stronger on average, overlap and wider top- k remain the strongest retrieval-side levers, and pairwise judges, QDMN refinement, and controller tuning still produce only modest practical differences while the oracle-pool gap stays large. The paper can therefore end on a cleaner final statement: the spiral story is now less pathological but still unresolved, whereas the selector bottleneck under ambiguity is robust across benchmark variants and remains the most direct next target.

Run X then adds the most targeted selector diagnostic so far by holding retrieval fixed and binning queries by shortlist margin. This makes the conditional structure of the problem explicit. In high-margin cases, nearly every practical method is already close to the oracle, which means those queries are effectively solved once retrieval is strong enough. In low-margin cases, however, the oracle gap remains large and current practical methods separate only modestly, with adaptive control or QDMN-style refinement helping only in narrow slices rather than across the board. The paper can therefore sharpen its ending one final time: the unresolved bottleneck is not generic “better judging” in the abstract, but specifically better discrimination among low-margin, high-ambiguity candidates inside an already exposed pool.

Run Y then broadens the placement question across several synthetic regimes and makes the current spiral weakness hard to dismiss as a one-off benchmark artifact. Random placement not only wins on average in every tested case, but also produces less concentrated root geometry, higher assignment entropy, more nonempty roots, and better coverage than either spiral variant. The orthonormalized spiral is still meaningfully better than the old spiral map, but it remains structurally more redundant than random and never overtakes it. The paper can therefore state a narrower architectural conclusion: before asking whether a stronger judge can exploit spiral placement, future work first has to produce a spiral-style root map that is not intrinsically coverage-poor relative to a random baseline.

Run AA finally asks whether the strongest current recipe improves once scale is pushed under matched budget. The answer is no for the present implementation: random remains stronger than `spiral_pca`, practical judges remain tightly clustered far below the oracle, and extra runtime does not buy a new end-to-end advantage. The most defensible final reading is therefore cautious rather than fatalistic. The scale result is substantial evidence against the current recipe, but not yet decisive evidence against the broader structured-memory idea, because root lifting, assignment geometry, and within-pool scoring are still plausibly limiting the implementation before the hypothesis has received its strongest test.